



Technische
Universität
Braunschweig

BACHELORARBEIT

Algorithmische Methoden zu gradbeschränkten Triangulierungen

Fabian Alich

5310894

Algorithmik

Institut für Betriebssysteme und Rechnerverbund
Abteilung Algorithmik

Erstprüfer

Prof. Dr. Sándor Fekete

Institut für Betriebssysteme und Rechnerverbund
Abteilung Algorithmik

Zweitprüfer

Prof. Dr. rer. nat. Roland Meyer

Institut für Theoretische Informatik

Betreuer

Dr. Phillip Keldenich

Michael Perk

24. August 2025

Statement of Originality

This thesis has been performed independently with the support of my supervisor/s. To the best of the author's knowledge, this thesis contains no material previously published or written by another person except where due reference is made in the text.

Braunschweig, 24. August 2025

Aufgabenstellung

Eine Triangulierung einer Punktmenge ist die Zerlegung ihrer konvexen Hülle in nicht überlappende Dreiecke. Triangulierungen spielen eine zentrale Rolle in der algorithmischen Geometrie und finden vielfältige Anwendungen, etwa in der Computergrafik und geographischen Informationssystemen (GIS). Im Rahmen seiner Bachelorarbeit soll Herr Alich das neuartige kombinatorische Optimierungsproblem Degree-Constrained Triangulations (DCT) untersuchen. Bei diesem Problem geht es darum, für eine gegebene Punktmenge eine Triangulierung zu finden, bei der – zusätzlich zu den klassischen Anforderungen – die Knotengrade (also die Anzahl der angrenzenden Dreiecke bzw. Kanten pro Punkt) gewissen Beschränkungen unterliegen. Ziel der Arbeit ist es, geeignete Verfahren zur Lösung dieses Problems zu entwickeln, zu implementieren und zu evaluieren. Dabei können neben exakten Verfahren (z.B. auf Basis von Integer Programming, Constraint Programming oder Branch-and-Bound) auch heuristische Ansätze (z.B. Greedy- oder Metaheuristiken) betrachtet werden. Von besonderem Interesse sind außerdem Methoden zur Fixierung oder zum Ausschluss bestimmter Kanten, um die Lösungsräume gezielt einzuschränken sowie die Generierung von leichten und schweren Instanzen für das Problem.

Abstract

This thesis examines the problem of degree constrained triangulations (DCT) where each vertex in a triangulation must have a given degree. Several exact and heuristic approaches were developed and implemented. These include edge based and triangle based encodings. In addition, strategies for fixing and excluding edges and systematic methods for generating instances for feasible and infeasible cases were introduced.

The experimental evaluation presents a comparison of different solvers such as PySAT, CaDiCaL, OR-Tools and Gurobi as well as various modeling techniques with respect to runtime and success rate. The results show that OR-Tools achieves the best results overall. All methods are only able to solve instances with a relatively small number of vertices. Optimization based formulations provide approximate solutions in infeasible or more complex cases.

The thesis shows that degree constrained triangulations can be effectively modeled and solved for small input sizes. OR-Tools proves to be the most effective solver. The results provide a foundation for further research on scaling to larger instances and developing improved heuristics.

FA: Ka wie ich das hier formatieren soll.

1. Deutsch

Diese Arbeit behandelt das Problem der gradbeschränkten Triangulationen (Degree Constrained Triangulations, DCT), bei denen jedem Knoten einer Triangulation ein vorgegebener Grad zugeordnet wird. Hierfür wurden exakte und heuristische Lösungsansätze entwickelt und implementiert. Dazu zählen kantenbasierte und dreiecksbasierte Kodierungen. Zusätzlich wurden Strategien zum Fixieren und Ausschließen von Kanten sowie systematische Verfahren zur Instanzgenerierung für lösbare und unlösbare Fälle entworfen.

Die experimentelle Untersuchung umfasst einen Vergleich verschiedener Solver wie PySAT, CaDiCaL, OR-Tools und Gurobi sowie unterschiedlicher Modellierungstechniken im Hinblick auf Laufzeit und Erfolgsquote. Die Ergebnisse zeigen, dass OR-Tools insgesamt die besten Resultate liefert. Alle Ansätze können jedoch nur Instanzen mit einer geringen Anzahl an Knoten lösen. Optimierungsbasierte Formulierungen liefern in unlösbaren oder komplexeren Fällen approximative Lösungen.

Die Arbeit zeigt, dass gradbeschränkte Triangulationen für kleine Eingabegrößen effizient modelliert und gelöst werden können. OR-Tools erweist sich dabei als leistungsfähigster Ansatz. Die Ergebnisse bilden eine Grundlage für weitere Forschung, insbesondere zur Skalierung auf größere Instanzen und zur Entwicklung verbesserter Heuristiken.

Inhaltsverzeichnis

1. Deutsch	vii
1. Einleitung	3
1.1. Das Problem	3
1.2. Ergebnisse dieser Arbeit	4
1.3. Related Work	5
1.3.1. Delaunay Triangulierung	5
1.3.2. Kantenflips in Triangulations	6
1.3.3. Degree constrained minimum spanning tree	7
2. Definitionen	11
3. Lösungsansätze	15
3.1. Entscheidungsproblem	15
3.1.1. Kantenansatz	15
3.1.2. Dreiecksansatz	21
3.2. Optimierungsproblem	23
3.3. <i>Conflict-driven clause learning</i>	25
3.3.1. <i>Unit-Klauseln</i>	25
3.3.2. Konflikte	26
4. Instanzgenerierung	29
4.1. Ja - Instanzen	29
4.1.1. Delaunay	29
4.1.2. Delaunay mit zufälligen Flips	29
4.1.3. Zufällige Kantenerzeugung	30
4.1.4. Iteration	30
4.1.5. Greedy	30
4.2. Nein-Instanzen	30
4.3. Theoretische Mehrere Lösungen	31
5. Solver	33
5.1. Generelle Implementierungen	33
5.2. PySAT	34
5.3. OrTools	34
5.4. Gurobi	35
5.5. Simplified CDCL Solve	36

6. Auswertung	37
6.1. Versuchsumgebung	37
6.2. Permutation	38
6.3. Mehrmalige Durchläufe	38
6.4. SAT Gradbeschränkung	38
6.5. SAT Solver Vergleich	40
6.6. Vergleich der Modellierungstechniken	41
6.7. Kanten vorab berechnen	41
6.8. Größerer Timeout	42
6.9. Ja/Nein getrennt	43
6.10. Zielfunktion	43
6.11. <i>Propagation</i> CDCL	45
6.11.1. Visualisierung	45
6.11.2. Relative Anzahl Erfolgreicher <i>Propagation</i>	46
6.12. Schwierigkeit der Insanzen	47
6.12.1. Anzahl Lösungen	47
6.12.2. Gradverteilung	48
6.12.3. Verteilung der Kantenlängen	49
6.12.4. Zusammenfassung	50
7. Conclusion (and Future Work)	51
A. Permutation	55
B. Mehrmalige Durchläufe	59
C. SAT Degree Auswertung	63
D. Alle Parameter	65
E. Anzahl Lösungen	69
F. größerer Timeout	71
G. Ja/Nein getrennt	73
H. Zielfunktion	75
I. Theoretische Fixierung von Kanten	77

Abbildungsverzeichnis

1.1. Eine Ja-Instanz und eine Nein-Instanz	4
1.2. Delauney Beispiel	6
1.3. Beispiel für <i>edge move</i> , <i>edge rotation</i> und <i>edge slide</i> aus [5, p. 70]	7
2.1. Ein Kantenflip in einer Triangulation	12
3.1. Gesonderte Betrachtung von Knoten auf der Hülle mit Grad zwei	16
3.2. zwei Beispielinstanzen an denen Kanten ausgeschlossen werden können . .	17
3.3. Kantenausschluss durch Schnittkanten	18
3.4. Ein Vollständiger Graph mit allen möglichen Triangulierung.	19
3.5. Ein Baum welcher die möglichen Entscheidungen eines SAT-Solvers darstellt. 26	
4.1. Visualisierung der fünf verschiedenen Instanztypen.	31
4.2. Eine Ja-Instanz mit zwei möglichen Lösungen.	32
6.1. Vergleich von Kardinalitätscodierungen für SAT-Solver	39
6.2. Vergleich verschiedener SAT-Solver	40
6.3. Vergleich aller Solver	42
6.4. In diesem Experiment wird die theoretische Fixierung von Kanten untersucht. 43	
6.5. Zielfunktion mit Nein-Instanz	44
6.6. Eine Teilinstanz in einem CDCL Solver	45
6.7. Anzahl Erfolgreichen <i>Propagation</i>	46
6.8. zwei verschiedene Lösungen für eine Instanz	47
6.9. In dieser Abbildung ist die Gradverteilung der Knoten pro Instanz zu sehen. Dabei wird auf der X-Achse der Grad und auf der Y-Achse die Häufigkeit der Knoten mit diesem Grad dargestellt.	48
6.10. In dieser Abbildung ist die Verteilung der Kantenlängen pro Instanz zu sehen.	49
A.1. Mehrfache Durchläufe mit verschiedenen Permutation für Gurobi vergleichen. 55	
A.2. Mehrfache Durchläufe mit verschiedenen Permutation für OrTools verglei- chen.	56
A.3. Mehrfache Durchläufe mit verschiedenen Permutation für PySAT vergleichen. 57	
B.1. Mehrfache Durchläufe mit gleichen Bedingungen für Gurobi vergleichen. . 59	
B.2. Mehrfache Durchläufe mit gleichen Bedingungen für OrTools vergleichen. 60	
B.3. Mehrfache Durchläufe mit gleichen Bedingungen für PySAT vergleichen. . 61	

Abbildungsverzeichnis

C.1. Vergleich von Kardinalitätscodierungen in eine KNF.	64
D.1. Eval1 Dreiecke	65
D.2. Die besten Bedingungen für PySAT finden	66
D.3. Die besten Bedingungen für OrTools finden	67
D.4. Die besten Bedingungen für Gurobi finden	68
F.1. In diesem Experiment wurde das Zeitlimit von 5 Minuten auf 30 Minuten erhöht.	71
G.1. Es wird untersucht ob Solver bei Ja-Instanzen oder bei Nein-Instanzen besser sind.	73
H.1. Eine Ja-Instanz als Optimierungsproblem betrachten	75
I.1. In diesem Experiment wird die theoretische Fixierung von Kanten untersucht.	77

Tabellenverzeichnis

E.1. Anzahl der Lösungen pro Instanz	69
--	----

FA: Passen die ersten Titel so? Bin sehr unsicher wie ich die zusammenfassen soll.

1. Einleitung

Triangulierungen sind ein zentraler Bestandteil der algorithmischen Geometrie. Eine Triangulierung zerlegt die konvexe Hülle einer Punktmenge in Dreiecke. In dieser Arbeit wird das klassische Problem um eine zusätzliche Bedingung erweitert. Für jeden Knoten wird ein Sollgrad vorgegeben.

Die gradbeschränkte Triangulierung (DCT) untersucht, ob es für eine gegebene Punktmenge eine Triangulierung gibt, bei der jeder Knoten einen vorgegebenen Grad besitzt. Das bedeutet, dass für jeden Knoten eine bestimmte Anzahl an inzidenten Kanten festgelegt ist. Diese Erweiterung ist nicht nur theoretisch interessant, sondern hat auch praktische Anwendungen. So kann beispielsweise in Sensornetzwerken die Verbindungs-dichte zwischen Knoten gezielt gesteuert werden. Auch in Logistikproblemen lassen sich vorgegebene Kapazitäten an Knoten berücksichtigen.

Das Ziel dieser Arbeit ist es, Verfahren zur Lösung des DCT zu untersuchen. Neben der Betrachtung als Entscheidungsproblem, bei dem Methoden wie Integer Programming, Constraint Programming oder SAT-Solver eingesetzt werden, wird das Problem auch als Optimierungsproblem betrachtet. Ein Schwerpunkt liegt auf dem Fixieren und Ausschließen von Kanten, um die Suche nach Lösungen zu beschleunigen. Im gleichen Zusammenhang werden verschiedene Methoden zur Instanzgenerierung vorgestellt.

Die Arbeit beginnt mit grundlegenden Definitionen und relevanten theoretischen Grundlagen. Anschließend werden verschiedene Lösungsansätze präsentiert und eine mögliche Implementierung beschrieben. Mit dieser Implementierung und zuvor generierten Instanzen werden die unterschiedlichen Ansätze evaluiert. Der Fokus liegt dabei auf der Laufzeit und dem Finden des besten Lösungsansatzes. Abschließend werden die generierten Instanzen hinsichtlich ihrer Schwierigkeit untersucht.

1.1. Das Problem

Das in dieser Arbeit betrachtete Problem sind gradbeschränkte Triangulierungen. Es wird untersucht, ob für eine gegebene Punktmenge eine Triangulierung existiert, bei der jeder Knoten einen vorgegebenen Grad besitzt. In Abbildung 1.1 sind zwei Instanzen mit Lösungen dargestellt.

Die Zahlen an den Knoten geben die jeweiligen Sollgrade an. Die obere Instanz ist eine Ja-Instanz, da die Sollgrade mit der Triangulierung eingehalten werden können. Bei der unteren Instanz, einer Nein-Instanz, ist mit den gegebenen Sollgraden keine Triangulierung möglich.

Eine Praktische Anwendung dieses Problems könnte zum Beispiel in der Computergrafik genauer in der Erzeugung von Netzen für die Modellierung von Oberflächen genutzt

1. Einleitung

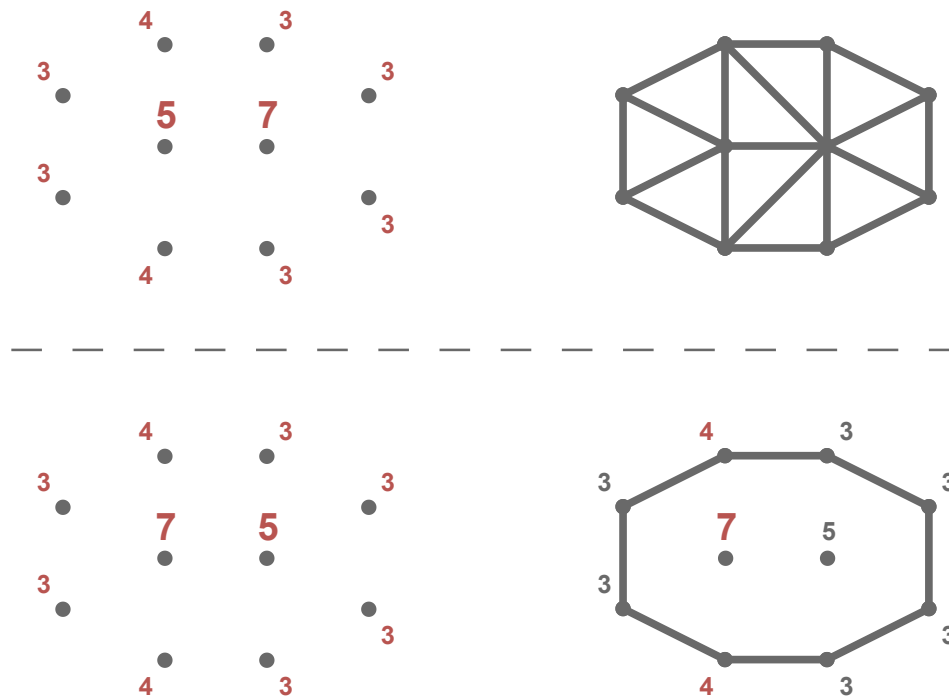


Abbildung 1.1.: Eine Ja-Instanz (oben) und eine Nein-Instanz (unten). Die Zahlen an den Knoten sind die Sollgrade. Bei der Nein-Instanz können die Sollgrade mit einer Triangulierung nicht eingehalten werden.

werden.

Durch den Sollgrad lässt sich die Dichte oder der Platzverbrauch in bestimmten Bereichen gezielt steuern. Analog kann in der Nachrichtentechnik ein DCT verwendet werden, um die Verbindungsdichte zwischen Knoten in einem Sensornetzwerk zu regulieren. Auch beim Transport von Gütern kann DCT genutzt werden. Der Sollgrad gibt dann an, wie viele Güter an jedem Ort vorhanden sind und weiterbewegt werden sollen.

1.2. Ergebnisse dieser Arbeit

In der Arbeit wurden zwei Hauptmodellierungen für DCT entwickelt, ein Kantenansatz und ein Dreiecksansatz. Beide Ansätze wurden sowohl als mathematische Modelle für Integer- beziehungsweise Constraint-Programming als auch als KNF-Formulierungen umgesetzt. Ergänzend wurden Heuristiken wie flip-basierte Verfahren entwickelt, die durch lokale Modifikationen versuchen, eine gültige Triangulierung zu erzeugen. Um den Lösungsprozess direkt zu optimieren wurde eine *Propagation* für einen CDCL-Solver implementiert.

Für die Evaluation wurden verschiedene Instanzentypen generiert, darunter zufällige

Punktmengen, Delaunay-basierte Instanzen und Greedy Ansätze.

Für die Lösung der Modelle wurden verschiedene Solver evaluiert. Dabei wurde PySAT für die KNF-Formulierungen verwendet und OR-Tools sowie Gurobi für die mathematischen Modelle. Bei diesen zeigte sich das OrTools Knotengrößen bis zu **170** Knoten Lösen konnte.

1.3. Related Work

In diesem Abschnitt werden verschiedene wissenschaftliche Arbeiten aus dem Themenbereich betrachtet. Dies dient dazu, einen Überblick über den aktuellen Entwicklungsstand zu geben. Es werden drei Bereiche betrachtet. Der erste Bereich behandelt die *Delaunay Triangulierung*. Diese zählt zu den bekanntesten Triangulationen und bietet einen grundlegenden Einblick in das Thema. Der zweite Bereich umfasst *Flips in Triangulationen*. Hier wird untersucht, wie sich Triangulierungen durch lokale Änderungen gezielt modifizieren lassen. Der dritte Bereich befasst sich mit dem Problem des *degree constrained minimum spanning tree*. Dabei werden insbesondere Ansätze zur Einhaltung von Gradbeschränkungen betrachtet.

Erwähnenswert ist, dass Sharir et al. [19] den minimalen und maximalen Grad in einer Triangulierung untersucht haben. Darüber hinaus haben Datta et al. [6] und Gewali et al. [11] sich mit Triangulierungen beschäftigt, bei denen jeder Knoten einen geraden Grad besitzt. Da diese beiden Themenbereiche keinen direkten Mehrwert für die vorliegende Arbeit bieten, werden sie hier lediglich der Vollständigkeit halber namentlich erwähnt.

1.3.1. Delaunay Triangulierung

Als Grundlage für den Algorithmus kann die *Delaunay-Triangulierung* betrachtet werden. Diese liefert eine Triangulierung, die möglichst gleichschenklige und gleichwinklige Dreiecke verwendet [17]. Dies wird erreicht, indem der minimale Winkel maximiert wird. Im Buch von de Berg et al. [8] wird diese Variante der Triangulierung ausführlich behandelt.

Triangulierungen eignen sich durch ihre Struktur besonders gut zur Darstellung von Höhendaten im zweidimensionalen Raum (siehe Abbildung 1.2). Dabei kann genutzt werden, dass die verwendeten Dreiecke unterschiedliche Innenwinkel und Kantenlängen besitzen.

Das Grundproblem besteht darin, dass die Höhendaten nur an bestimmten Punkten bekannt sind und die restlichen Werte daher geschätzt werden müssen. Dies kann durch die Delaunay-Triangulierung gut gelöst werden, da die Dreiecke aufgrund ihrer gleichschenkligen Eigenschaft visuell geeignet sind, die fehlenden Daten zu ergänzen.

Es werden auch Formeln für die Anzahl der Kanten $|E|$ und Dreiecke $|F|$ in einer Triangulierung angegeben. Für eine Punktemenge mit n Punkten und k Punkten auf der konvexen Hülle gilt:

$$\begin{aligned} |E| &= 3n - 3 - k \\ |F| &= 2n - 2 - k \end{aligned}$$

1. Einleitung

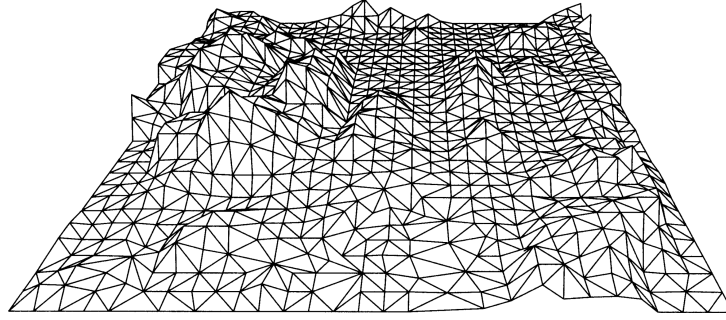


Abbildung 1.2.: Ein Beispiel für Höhendaten durch eine Triangulierung aus [8, p. 183]

1.3.2. Kantenflips in Triangulations

Kantenflips oder kurz Flips sind Möglichkeiten, um Kanten in einer Triangulierung auszutauschen. Wie oben erwähnt, müssen Triangulierung eine feste Anzahl an Kanten besitzen [8]. Dementsprechend können keine Kanten hinzugefügt oder entfernt werden da dies die Gradsumme der Triangulierung verändern würde. Daraus folgt, dass für jede entfernte Kante eine andere Kante eingefügt werden muss. Eine besondere Art dieser Umstrukturierung von Kanten sind sogenannte Flips. Hurtado et al. [12] zeigten, dass jede Triangulierung mindestens

$$\frac{n - 4}{2}$$

flipbare Kanten besitzt, wobei n die Anzahl der Knoten bezeichnet. Laut der wissenschaftliche Arbeit *Flips in planar graphs* [5] lässt sich jede mögliche Triangulierung in jede beliebige andere Triangulierung überführen, indem die notwendigen Flips durchgeführt werden. Daraus folgt, dass, wenn DCT möglich ist, diese durch Flips konstruiert werden kann. Dieser Ansatz wird in Abschnitt 3.2 nochmal näher betrachtet. In Triangulierungen gibt es nur eine Art von Flips, die sogenannten diagonalen Flips. Bei diesen wird ein konvexes Viereck betrachtet und die Diagonale durch die andere Diagonale ersetzt. Genauer wird dies in Definition 2.6 beschrieben.

Wenn man Flips außerhalb von Triangulierungen betrachtet, gibt es verschiedene Arten von Flips. Drei beispielhafte Flips sind in Abbildung 1.3 dargestellt. Zum einen der *edge move*, hierbei handelt es sich um das einfache Entfernen und anschließende Einfügen einer Kante. Etwas stärker eingeschränkt ist die *edge rotation*, bei der ein Endpunkt der Kante unverändert bleibt. Die Rotation erfolgt also nur um einen Knoten. Die dritte Variante, der *edge slide*, schränkt die *edge rotation* weiter ein. Hier darf der veränderte Knoten nur ein Nachbar des ursprünglichen Knotens sein.

Mann kann Flips noch weiter unterteilen. In sogenannte k -Flips bei diesen wird noch der Anzahl der veränderten Kanten gruppiert. Dabei werden k Kanten entfernt und durch k neue Kanten ersetzt. Die bisher genannten Flips entsprechen somit 1-Flips.

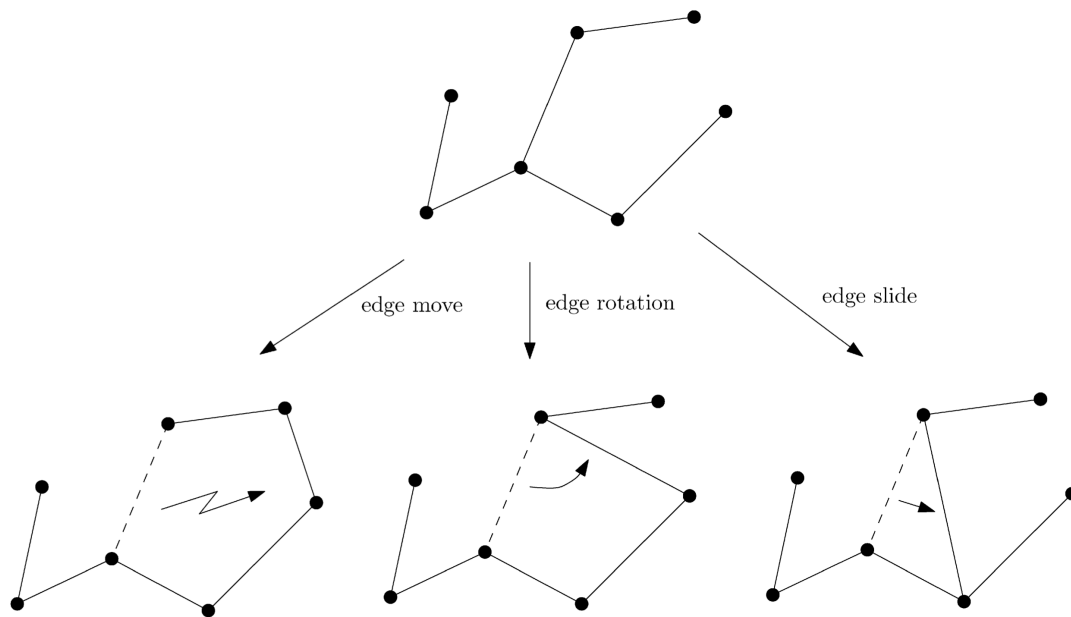


Abbildung 1.3.: Beispiel für *edge move*, *edge rotation* und *edge slide* aus [5, p. 70]

1.3.3. Degree constrained minimum spanning tree

Eine Gradbeschränkung gibt es auch beim Minimum Spanning Tree. Hier hat jede Kante ein vorgegebenes Gewicht, welches im Spanning Tree minimiert werden soll. Das Besondere am *degree constrained minimum spanning tree* ist, dass ein Minimum Spanning Tree erzeugt werden soll, welcher an jedem Knoten einen maximalen Grad von d hat. Dabei wird d global für das Problem vorgegeben.

Es wurden verschiedene Ansätze von Joshua Knowles und David Corne [13] vorgestellt, welche das Problem lösen sollen. Der erste Ansatz ignoriert zu Beginn den Aspekt der Minimalität des Problems. Es werden Kanten hinzugefügt, welche die Gradbeschränkung nicht verletzen. Im weiteren Verlauf wird versucht, die Anzahl der Kanten zu minimieren.

Ein anderer Ansatz nutzt aus, dass der minimale Spannbaum ohne Gradbeschränkung in polynomialer Zeit gelöst werden kann. Dieser Ansatz erzeugt zuerst einen minimalen Spannbaum, welcher die Gradbeschränkung ignoriert. Wenn eine Lösung gefunden wurde, werden den Kanten neue Gewichte zugeteilt. Diese neuen Gewichte werden mit folgender Formel berechnet,

$$weight + fault \cdot max \cdot \frac{(weight - min)}{(max - min)}$$

wobei *weight* das aktuelle Gewicht der Kante ist. Die Variable *fault* hat den Wert, $\{0, 1, 2\}$ abhängig davon, wie viele Knoten an der Kante gerade die Gradbeschränkung verletzen. Die Variablen *min* und *max* bezeichnen das kleinste beziehungsweise größte Gewicht aller Kanten. Danach wird erneut ein minimaler Spannbaum gebildet, wobei durch das neue Gewicht nun Kanten bevorzugt werden, welche keine Gradbeschränkung

1. Einleitung

verletzen. Diese beiden Schritte können so lange wiederholt werden, bis eine valide Lösung gefunden wurde oder eine maximale Anzahl an Iterationen erreicht wurde. Der Nachteil bei diesem Ansatz ist, dass Lösungen fälschlicherweise als *nicht möglich* bewertet werden können aufgrund der maximalen Iterationen.

Ein weiterer Ansatz wird als *Hillclimbing* bezeichnet. Dieser löst den Nachteil des *degree constrained minimum spanning tree* nicht, wird jedoch zur Suche nach neuen Knoten in einem anderen Algorithmus verwendet. In diesem Ansatz werden lokal neue Knoten gesucht, die bessere Ergebnisse liefern. Dies könnte auf die *Degree constrained Triangulierung* angewendet werden, um neue Kanten zu finden. Der Ansatz erzeugt keine eigene Lösung, sondern verbessert eine bestehende. Zunächst wird ein zufälliger Knoten aus einer Lösung ausgewählt. Danach werden weitere mögliche Knoten in der Nähe des ursprünglichen Knotens gesucht. Bei diesen Knoten wird überprüft, ob ein Knotentausch gleich gute oder bessere Ergebnisse liefern. Falls dies der Fall ist, wird der Knoten ausgetauscht, andernfalls wird der gefundene Knoten ignoriert. Dieser Ansatz soll lokal gute Ergebnisse liefern, bringt jedoch aufgrund seines *greedy*-Verhaltens keine globalen Verbesserungen. Eine mögliche Verbesserung der lokalen Problematik stellt *multistart hillclimbing* dar. Bei diesem Verfahren wird nur ein Punkt in der Nähe getestet. Ist dieser nicht gleich oder besser, wird ein neuer zufälliger Punkt ausgewählt.

Der nächste Ansatz wird *simulated annealing* genannt. Auch hier werden nur neue Knoten gesucht, um bestehende Lösungen zu verbessern. Das Ziel des Ansatzes ist es, das Problem mit einer globalen Verbesserung zu lösen, indem zu Beginn viele Veränderungen zugelassen werden und im Verlauf immer genauere Verbesserungen erfolgen. Auch hier wird ein zufälliger Knoten ausgewählt und es werden weitere mögliche Knoten in der Nähe gesucht. Bei diesem Ansatz werden jedoch neue Knoten gemäß der folgenden Formel übernommen:

$$p\{\text{accept } j\} = \begin{cases} 1 & \text{if } f(j) \leq f(i) \\ \exp\left(\frac{f(i)-f(j)}{c}\right) & \text{sonst} \end{cases}$$

Dabei ist $p\{\text{accept } j\}$ die Wahrscheinlichkeit, dass ein Knoten übernommen wird. Die aktuelle Lösung ist i und die mögliche neue Lösung ist j . Die Funktion f gibt hierbei die Kosten der Lösung an. Der Parameter c wird zu Beginn sehr hoch gesetzt und mit der Zeit verringert. Die Veränderung von c bewirkt die gewünschte Änderungsrate. Der Ansatz endet, wenn ein finales c_f erreicht wird, also wenn $c = c_f$.

Der letzte Ansatz beschreibt das übliche Verfahren eines Evolutionsalgorithmus. Dabei werden verschiedene Lösungen erzeugt und bewertet. Die besten Lösungen werden ausgewählt und auf Basis dieser Lösungen werden durch Mutation neue Lösungen generiert.

Krishnamoorthy et.al. [15] hat Evolutionsalgorithmen noch mal genauer betrachtet. Der erste Schritt ist es eine Codierung zu wählen, welche nur zulässige Spannbäume darstellen kann. Sonst kann es passieren, dass von den erstellten Lösungen viele nicht genutzt werden können und somit nicht relevant sind. Wenn eine passende Codierung gewählt wurde, können dann nur noch der Grad und das Gewicht betrachtet werden.

FA: Sollte ich hier lieber bei Parent und Child bleiben oder das so eingedeutscht nutzen?

Eine Möglichkeit wie Mutationen gebildet werden können, ist die *Crossover* Methode. Dabei existieren zwei *Eltern* Lösungen P_1 und P_2 . Aus diesen werden

$$2 \cdot (n - 2)$$

verschiedene *Kinder* erzeugt, da es $n - 1$ Veränderungsmöglichkeiten gibt. Anschließend werden die *Kinder* bewertet und sortiert. Das beste *Kind*, das P_1 am ähnlichsten ist, wird ausgewählt. Ist dieses *Kind* besser als P_1 , ersetzt es P_1 . Für P_2 wird ein *Kind* nach dem gleichen Verfahren ausgewählt.

Eine weitere Möglichkeit ist die *Breeding* Methode. Dabei existiert ein Pool aus Lösungen. Zufällig werden Paare gebildet, die jeweils zwei *Kinder* erzeugen. Von diesen *Kindern* ersetzt das bessere *Kind* das schlechtere *Elternteil*. Die Größe des Pools kann so angepasst werden, dass alle möglichen Veränderungen berücksichtigt werden.

Beide Methoden sollen im Durchschnitt alle 10 Generationen die beste Lösung mit allen anderen Lösungen verglichen haben. Als mögliche Iterationsanzahl wird folgende Formel verwendet,

$$\frac{50 \cdot n}{\bar{b}}$$

dabei bezeichnet $\bar{b}$ die durchschnittliche Gradbeschränkung. Diese Formel sorgt dafür, dass große Instanzen mehr Iterationen erhalten, während leichtere Instanzen (mit höherem Grad) schneller bearbeitet werden.

Sandor P. Fekete et al. [10] betrachten ebenfalls das *degree constrained minimum spanning tree* problem. Hierbei wird ein *Flow based Algorithmus* verwendet. Da dieser Ansatz wahrscheinlich nicht gut auf Triangulations übertragbar ist, wird er hier nur kurz behandelt.

Im praktischen Teil des Papers werden zwei Algorithmen vorgestellt. Diese erhalten einen minimalen Spannbaum und verändern ihn so, dass er eine Gradbeschränkung einhält. Dabei wird darauf geachtet, dass sich das Gewicht des Spannbaums nicht stark verschlechtert. Für diese Algorithmen werden Hauptoperationen namens *Adoptions* eingeführt. Diese funktionieren ähnlich wie Flips nur für einen Spannbaum. Es handelt sich also um Operationen, die eine zulässige Veränderung bewirken. Die beiden Algorithmen liefern eine Liste von Adoptionen, welche die gewünschte Änderung erzeugen. Der erste Algorithmus nutzt eine fraktionale Lösung und eine Greedy Strategie, um die Adoptionen zu finden. Der andere Algorithmus beschränkt die betrachteten Kanten auf jene des gegebenen Spannbaums. So können die Algorithmen in polynomialer Zeit eine Lösung finden, die eine Gradbeschränkung einhält.

Eine Abwandlung des minimalen Spannbaums mit Gradbeschränkung wurde von Ana Maria de Almeida et al. [7] untersucht. Dabei erweitern sie das Problem um eine Funktion, welche für jeden Knoten einen Mindestgrad vorgibt. Dieses Problem wird als *Min-Degree Constrained Minimum Spanning Tree* bezeichnet.

Es wurde das *k-Dimensional Matching Problem* verwendet, um zu zeigen, dass das *Min-Degree Constrained Minimum Spanning Tree* Problem NP-schwer ist. Zu erwähnen ist, dass dies in der dritten Dimension nicht gezeigt wurde. Allerdings soll das Problem

1. Einleitung

schwieriger sein als das *Degree Constrained Minimum Spanning Tree*, obwohl auch dieses Problem NP-schwer ist.

Auch in diesem Fall wird das Problem mit einer *Flow based Formulierung* betrachtet. Dabei werden zwei Formulierungen angegeben. Eine gerichtete und eine ungerichtete Formulierung. Die gerichtete Formulierung soll hierbei effizienter Ergebnisse liefern. Die Formulierungen wurden mit einem *Branch and Bound* Algorithmus getestet. Dabei wurde festgestellt, dass es bei der gerichteten Formulierung relevant ist, welcher Knoten als Wurzelknoten ausgewählt wird. Es konnte jedoch kein einheitlich optimaler Wurzelknoten für die Formulierungen festgestellt werden. Des Weiteren wurde häufig festgestellt, dass die Relaxierung keine guten Schranken lieferte. Dies konnte damit erklärt werden, dass die zugehörige Formel für den minimalen Spannbaum ohne Gradbeschränkung gleiche Ergebnisse lieferte.

2. Definitionen

In diesem Kapitel werden die Definitionen vorgestellt, die für diese Arbeit benötigt werden. Dabei werden insbesondere grundlegende Begriffe erläutert, die in den folgenden Kapiteln verwendet werden.

Zu Beginn werden Überschneidungen zwischen Kanten betrachtet.

Definition 2.1 (Überschneidung). Zwei Kanten gelten als überschneidend, wenn sie sich in der Ebene kreuzen. Im Folgenden bezeichnet die Funktion $I : (\mathbb{Q}^2 \times \mathbb{Q}^2) \rightarrow \mathcal{P}(\mathbb{Q}^2 \times \mathbb{Q}^2)$ die Menge aller Kanten, die eine gegebene Kante schneiden.

Im nächsten Schritt wird die Überschneidung von Dreiecken definiert, da diese in späteren Ansätzen relevant werden.

Definition 2.2 (Überschneidung Dreiecke). Zwei Dreiecke gelten als überschneidend, wenn sie mindestens einen inneren Punkt gemeinsam haben. Das bedeutet, dass sich zwei Außenkanten kreuzen und sich nicht nur berühren dürfen. Im Folgenden bezeichnet die Funktion $I : (\mathbb{Q}^2 \times \mathbb{Q}^2 \times \mathbb{Q}^2) \rightarrow \mathcal{P}(\mathbb{Q}^2 \times \mathbb{Q}^2 \times \mathbb{Q}^2)$ die Menge aller Dreiecke, die ein gegebenes Dreieck schneiden.

Auf Grundlage der Überschneidungen können nun die grundlegenden Begriffe der Triangulierung und DCT definiert werden.

Definition 2.3 (Triangulierung). Eine Triangulierung einer Punktmenge $P \subset \mathbb{Q}^2$ ist ein maximaler kreuzungsfreier Graph auf P . Das bedeutet, es können keine weiteren Kanten hinzugefügt werden, ohne dass sich Kanten schneiden.

Für DCT wird zuerst der Grad eines Knotens definiert.

Definition 2.4 (Grad). Der Grad eines Knotens ist die Anzahl der anliegenden Kanten. Im Folgenden wird er durch die Funktion $\deg : \mathbb{Q}^2 \rightarrow \mathbb{N}$ angegeben.

Basierend auf dem Grad eines Knotens wird nun DCT eingeführt.

Definition 2.5 (Gradbeschränkte Triangulierung). Die gradbeschränkte Triangulierung (DCT) hat die gleichen Voraussetzungen wie eine Triangulierung. Zusätzlich gibt es eine Funktion $\deg^* : \mathbb{Q}^2 \rightarrow \mathbb{Q}$. Diese gibt den Grad an, den ein Knoten haben muss.

Im Folgenden wird die Durchführung des Kantenflips beschrieben, die eine wichtige Rolle bei der Umstrukturierung von Triangulierungen spielt.

2. Definitionen

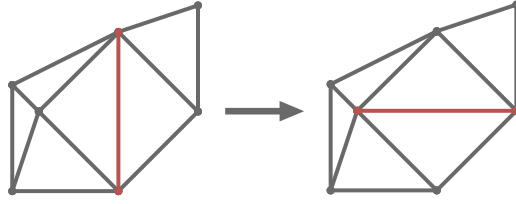


Abbildung 2.1.: Ein Kantenflip in einer Triangulation

Definition 2.6 (Kantenflip). Ein Kantenflip oder kurz Flip ist eine Operation, bei der eine Kante entfernt und eine andere hinzugefügt wird, sodass die Eigenschaften einer Triangulierung erhalten bleiben. In dieser Arbeit ausschließlich mit Triangulierungen gearbeitet dementsprechend wird nur der diagonale Flip betrachtet. Bei diesem wird die Diagonale eines konvexen Vierecks durch die andere Diagonale ersetzt. Ein beispielhafter Flip ist in Abbildung 2.1 zu sehen.

Im nächsten Schritt wird die konvexe Hülle mit Randpunkten definiert. Da die konvexe Hülle in dieser Arbeit um alle Randpunkte erweitert werden muss.

Definition 2.7 (Konvexe Hülle mit Randpunkten). Die konvexe Hülle einer Punktmenge ist die kleinste konvexe Menge, die alle Punkte enthält. Die Randpunkte sind die Punkte, die auf dem Rand der konvexen Hülle liegen, einschließlich solcher, die nicht an einer Ecke liegen. Die konvexe Hülle inklusive Randpunkte wird durch die Funktion $CH^* : \mathcal{P}(\mathbb{Q}^2) \rightarrow \mathcal{P}(\mathbb{Q}^2)$ definiert. In dieser Arbeit werden bei der konvexen Hülle immer die Randpunkte mit einbezogen.

Abschließend wird die konjunktive Normalform eingeführt, die im weiteren Verlauf für die Benutzung der SAT-Solver benötigt wird.

Definition 2.8 (Konjunktive Normalform). Eine Formel in konjunktiver Normalform (KNF) ist eine logische Formel, die als Konjunktion (UND-Verknüpfung) von Klauseln dargestellt wird, wobei jede Klausel eine Disjunktion (ODER-Verknüpfung) von Literalen ist. Ein Literal ist entweder eine Variable oder ihre Negation. Ein Beispiel für eine KNF ist

$$(x_1 \vee \overline{x_2}) \wedge (x_2 \vee x_3)$$

wobei x_1 , x_2 und x_3 Variablen sind. Eine Negation wird durch das Symbol $\overline{x}$ dargestellt.

Da in einigen Ansätzen die Orientierung von Punkten eine Rolle spielt, werden im Folgenden die relevanten Definitionen vorgestellt. Zunächst wird das Kreuzprodukt definiert, das die Orientierung von Vektoren beschreibt.

Definition 2.9 (Kreuzprodukt). Das Kreuzprodukt zweier Vektoren $\vec{a} = (a_1, a_2)$ und $\vec{b} = (b_1, b_2)$ in der Ebene ist ein Skalarwert, der die Orientierung der beiden Vektoren zueinander beschreibt. Es wird definiert als:

$$\vec{a} \times \vec{b} = a_1 b_2 - a_2 b_1$$

Das Kreuzprodukt kann anschließend verwendet werden, um die Orientierung von drei Punkten zu bestimmen.

Definition 2.10 (Rechtsknick / Linksknick). Ein Rechtsknick ist eine Abfolge von drei Punkten p_1, p_2 und p_3 , die im Uhrzeigersinn angeordnet sind. Ein Linksknick ist eine Abfolge von drei Punkten p_1, p_2 und p_3 , die gegen den Uhrzeigersinn angeordnet sind. Die Orientierung der Punkte kann durch das Kreuzprodukt der Vektoren $\overrightarrow{p_1p_2}$ und $\overrightarrow{p_2p_3}$ bestimmt werden. Wenn das Kreuzprodukt positiv ist, handelt es sich um einen Linksknick, wenn es negativ ist, um einen Rechtsknick. Ist das Kreuzprodukt null, liegen die Punkte auf einer Geraden.

Der Rechts- und Linksknick wird verwendet, um zu bestimmen, ob ein Punkt links oder rechts von einer Kante liegt.

Definition 2.11 (Links/Rechts von einer Kante). Ein Punkt p liegt links von einer Kante (p_1, p_2) , wenn die Punkte p_1, p_2 und p einen Linksknick bilden. Analog liegt ein Punkt p rechts von einer Kante (p_1, p_2) , wenn die Punkte p_1, p_2 und p einen Rechtsknick bilden.

3. Lösungsansätze

In diesem Kapitel werden die verschiedenen Ansätze erklärt mit denen das Problem gelöst werden kann. Im ersten Abschnitt wird das Problem als Entscheidungsproblem betrachtet. Also nur entschieden ob es eine Ja-Instanz oder eine Nein-Instanz ist. Danach wird das Problem als Optimierungsproblem betrachtet. Das bedeutet unabhängig von der Instanz eine möglichst gute Lösung zu finden. Im letzten Abschnitt wird das Problem während des Lösungsprozesses optimiert.

3.1. Entscheidungsproblem

In diesem Abschnitt wird das DCT als Entscheidungsproblem betrachtet. Das bedeutet dem Solver wird eine Instanz übergeben und dieser soll entscheiden, ob eine Lösung existiert oder nicht.

Unabhängig vom Ansatz muss für das DCT überprüft werden, ob die Summe der Sollgrade zur Anzahl der Kanten passt. Mit Hilfe der Gleichung

$$|E| = 3n - 3 - k$$

kann die Anzahl der benötigten Kanten in einer Triangulierung berechnet werden. Diese Anzahl kann dann in der Gleichung

$$|E| \cdot 2 \stackrel{!}{=} \sum \deg^*(i)$$

genutzt werden um zu überprüfen ob die Sollgrade eine Ja-Instanz ermöglichen.

Im folgenden, werden zwei verschiedene Ansätze beschreiben. Zum einen ein Kantenansatz und zum anderen ein Dreiecksansatz.

3.1.1. Kantenansatz

Bei diesem Ansatz werden Kanten betrachtet. Das bedeutet die DCT wird erreicht, indem bestimmte Bedingungen an die Kanten gestellt werden. Dafür werden zuerst zwei notwendige Bedingungen erklärt und dann weitere optionale, welche die Laufzeit verbessern könnten.

Notwendig Bedingungen

Da eine Triangulierung keine Überschneidungen enthalten darf, werden alle Kantenpaare berechnet. Von jedem dieser Paare darf dann nur eine Kante aktiv sein.

3. Lösungsansätze

Die zweite Bedingung ist die Gradbeschränkung. Für jeden Knoten i wird die Summe der inzidenten Kanten auf den Sollgrad $\deg^*(i)$ beschränkt. Um dies zu erreichen, wird für jeden Knoten festgelegt, dass die Summe der ausgehenden Kanten dem Sollgrad entspricht.

Kanten fixieren/ausschließen

Die erste und naheliegendste Bedingung ist die *fixierte Hülle*. Hierbei werden die Kanten der konvexen Hülle immer hinzugefügt. Diese dürfen hinzugefügt werden, da sie keine Schnittkanten besitzen. Würde eine dieser Kanten nicht hinzugefügt werden, wäre der Graph nicht maximal und somit keine Triangulierung. Da die Kanten der Hüllen in jeder möglichen Triangulierung vorhanden sind, verletzen sie auch nicht eine valide Gradbeschränkung.

Eine weitere optionale Bedingung ist zuvor berechnete Kanten zu fixieren oder auszuschließen. Eine Möglichkeit um Kanten zu fixieren ist immer drei aufeinander folgende

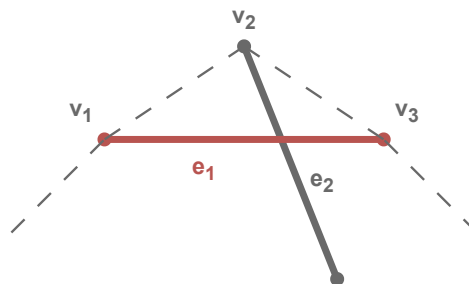


Abbildung 3.1.: Abhängig von dem Sollgrad von v_2 kann entschieden werden, ob e_1 aktiv oder nicht aktiv sein muss. Sollte $\deg^*(v_2) = 2$ sein, dann muss e_1 aktiv sein damit alle Kanten wie e_2 nicht aktiv sein können. Sollte $\deg^*(v_2) > 2$ sein, dann muss e_1 nicht aktiv sein damit der Sollgrad erreicht werden kann.

Knoten der Hülle zu betrachten. Ein mögliches Beispiel ist in Abbildung 3.1 zu sehen. Sollte $\deg^*(v_2) = 2$ sein, muss die Kante e_1 in der Triangulierung genutzt werden. Der Grund dafür ist, dass alle Schnittkanten von e_1 nicht aktiv sein dürfen, da sie den Grad von v_2 erhöhen würden. Analog dazu kann die Kante e_1 ausgeschlossen werden, sollte $\deg^*(v_2) > 2$ sein. Sonst kann der Sollgrad nicht erreicht werden. Ein $\deg^*(v_2) < 2$ ist nicht möglich in DCT.

Zwei weitere Bedingungen, um Kanten auszuschließen, basieren auf der Teilung der Punktmenge. Es können also alle Kanten betrachtet werden, welche beide Knoten auf der Hülle haben. Die folgenden beiden Bedingungen können genutzt werden, um diese Kanten auszuschließen. Sollte eine der Bedingungen wahr werden, so kann die Kante ausgeschlossen werden. Die Bedingungen werden für beide Hälften getestet, welche durch

die Teilung entstehen.

$$|E| \cdot 2 > \sum_{v \in V} \deg^*(v)$$

$$\forall v \in (V \setminus V_H) : \deg^*(v) > |V| - 1$$

Dabei sei V die Menge aller Knoten in der betrachteten Hälfte. Analog sei $V_H \subseteq V$ die Menge der Knoten auf der Hülle.

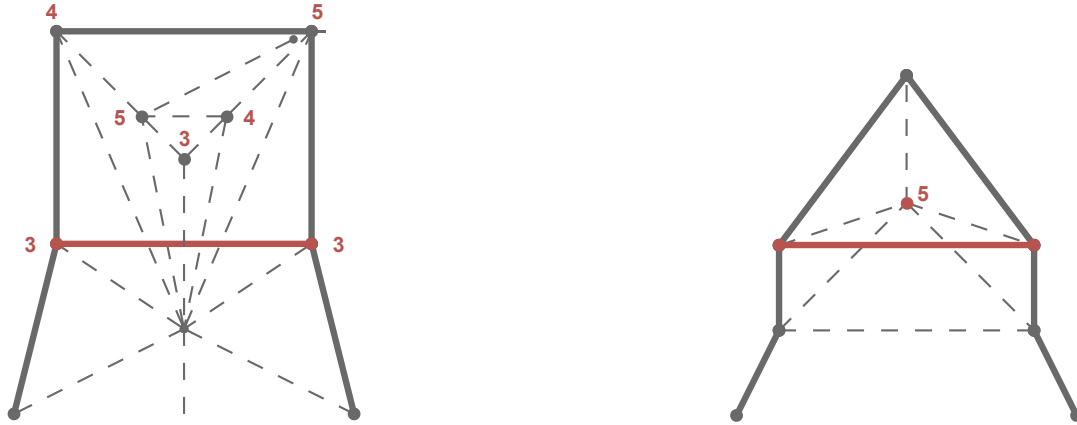


Abbildung 3.2.: Zwei Teilinstanzen, in welcher die rote Kante ausgeschlossen werden kann. Links da die Gradsumme die Anzahl der Kanten unterschreitet und rechts da der mittlere rote Knoten nicht genügend Kanten erzeugen könnte.

Zwei Beispiele sind in Abbildung 3.2 zu sehen. Dabei ist links eine Teilinstanz, in welcher die Anzahl der benötigten Kanten (28) größer als die Summe der Grade (27) ist. Auf der rechten Seite ist eine Teilinstanz in welcher der mittlere rote Knoten mit Grad 5 größer ist als die Anzahl der Knoten minus eins (3).

Eine weitere Möglichkeit, Kanten auszuschließen, besteht darin, ihre Schnittkanten zu betrachten. Wenn eine Kante e_1 zu viele Kanten eines Knotens v_1 blockiert, sodass dieser seinen Sollgrad nicht mehr erreichen kann, kann e_1 ausgeschlossen werden.

In Abbildung 3.3 ist zu erkennen, dass e_2 , e_3 und e_4 durch e_1 blockiert werden. Da v_1 jedoch einen Sollgrad von 3 besitzt und nur 5 mögliche Kanten existieren, kann v_1 mit der aktiven Kante e_1 seinen Sollgrad nicht erreichen.

Alternative Kanten

Eine andere mögliche Bedingung ist die *alternative Kantenbeschränkung*. Diese wurde von Sándor P. Fekete et.al. [9] übernommen. Diese Bedingung nutzt aus, dass der Graph maximal sein muss. Damit dies der Fall sein kann, muss für jede Kante gelten, dass entweder die Kante aktiv ist oder eine der Schnittkanten aktiv ist. Wie man in Abbildung 3.4

3. Lösungsansätze

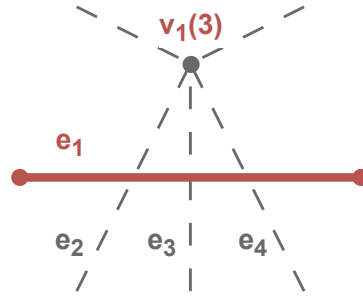


Abbildung 3.3.: In der Abbildung hat der Knoten v_1 einen Sollgrad von 3. Da die Kante e_1 drei von fünf möglichen Kanten blockiert, kann v_1 seinen Sollgrad nicht erreichen. Daher kann e_1 ausgeschlossen werden

sieht, ist e_1 die ausgewählte Kante und e_2 und e_3 die Schnittkanten. Weiter unten im Bild ist zu sehen, dass alle möglichen Triangulierungen mindestens eine der drei Kanten verwendet. Zu beachten ist allerdings das die Anzahl der Schnittkanten nicht gleich eins gesetzt werden darf, sondern nur auf mindestens eins. Wie man in Abbildung 3.4 sieht, können die Kanten e_2 und e_3 gleichzeitig aktiv sein.

Formulierungen

Im Folgenden werden zwei verschiedene Formulierungen beschrieben, die den Kantenansatz umsetzen. Zum einen eine mathematische Formulierung welche Formeln erstellt. Zum anderen eine Formulierung in konjunktiver Normalform (KNF). Für diese Formulierung wird angenommen, dass V die Menge aller Knoten und E die Menge aller möglicher Kanten sei, also der Kanten die keine Punkte schneiden. Für jede Kante $e \in E$ wird eine Bool Variable x_e erstellt. Diese Variable ist wahr, wenn die Kante e in der Triangulierung enthalten ist.

Mathematische Formulierung

Für die Überschneidungsbeschränkung darf für jedes Variablenpaaren nur eine Variable aktiv sein.

$$(x_{e_1}, x_{e_2}) \quad \forall e_1, e_2 \in E : e_2 \in I(e_1)$$

Dies kann mit folgender Formel umgesetzt werden.

$$x_{e_1} + x_{e_2} \leq 1 \quad \forall e_1, e_2 \in E : e_2 \in I(e_1)$$

Für die Gradbeschränkung wird die Summe der booleschen Variablen x_e für alle Kanten eines Knotens i auf den Sollgrad $\deg^*(i)$ beschränkt.

Dafür kann folgende Formel genutzt genutzt werden.

$$\sum x_{e_i} = \deg^*(i) \quad \forall i \in V$$

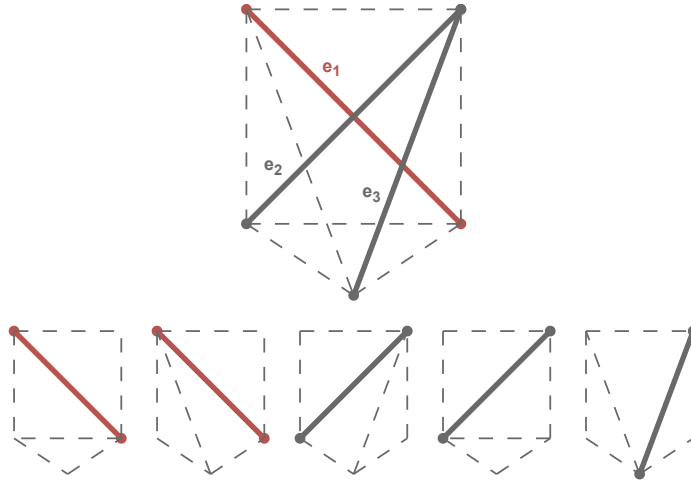


Abbildung 3.4.: Ein Vollständiger Graph mit allen möglichen Triangulierung. Die Kanten e_2 und e_3 sind Schnittkanten von e_1 . Es ist zu sehen, dass e_1 oder eine der Schnittkanten aktiv sein muss, damit eine Triangulierung möglich ist.

Wobei e_i alle Kanten für den Knoten i sind.

Für die *alternative Kantenbeschränkung* kann folgende Gleichung genutzt werden.

$$\sum_{j \in I(e)} x_j + x_e \leq 1 \quad \forall e \in E$$

Für die fixierung bzw. ausschließung der Kanten werden die zugehörigen Variablen auf *Wahr* oder *Falsch* gesetzt.

KNF Formulierung

Die Überschneidungen können sehr ähnlich mit folgender Formel umgesetzt werden:

$$\overline{x_{e_1}} \vee \overline{x_{e_2}} \quad \forall e_1, e_2 \in E : e_2 \in I(e_1)$$

Für die Gradbeschränkung ist, dass Problem, dass in KNF für eine Menge von Variablen nicht direkt festgelegt werden kann, wie viele von ihnen wahr sein müssen. Im folgenden werden zwei verschiedene Ansätze beschrieben, um dies doch zu erreichen. Der erste Ansatz nutzt Kardinalitätscodierungen aus einer Sammlung von Codierungen. Diese Sammlung wird von der Python Bibliothek PySAT bereitgestellt. Von diesen Codierungen kommen nur *sequential counters* [20], *sorting networks* [3], *cardinality networks* [1], *totalizer* [2], *modulo totalizer* [18], *modulo totalizer for k-cardinality* [16] und eine native Codierung in Frage. Die restlichen Codierungen bieten keine Möglichkeit um Kardinalitätsbeschränkungen größer als eins zu erzeugen.

3. Lösungsansätze

In Abschnitt 6.4 wird sich zeigen, dass *sequential counters* am besten geeignet ist. Daher wird dieser im Folgenden genauer erläutert. Der Ansatz der *sequential counters* wurde von Carsten Sinz [20] entwickelt. Er basiert auf einer Idee aus der Computerhardwareentwicklung. Zur Berechnung der Kardinalität k für die Klausel $(x_1 \vee x_2 \vee \dots \vee x_n)$ werden $n - 1$ Variablen s_i der Länge k in der unären Darstellung benötigt. Zusätzlich werden n Hilfsvariablen v_i verwendet. Für $i = 1$ werden s_1 und v_1 wie folgt definiert

$$\begin{aligned} s_{1,1} &\Leftrightarrow x_1 \\ s_{1,j} &\Leftrightarrow 0 && \text{für } 1 < j \leq k \\ v_1 &\Leftrightarrow 0 \end{aligned}$$

Für $i = n$ wird v_n wie folgt gesetzt

$$v_n \iff x_n \wedge s_{n-1,k}$$

Iterativ werden dann s_i und v_i für $i \in \{2, \dots, n - 1\}$ wie folgt definiert

$$\begin{aligned} s_{i,1} &\Leftrightarrow x_i \vee s_{i-1,1} \\ s_{i,j} &\Leftrightarrow x_i \wedge s_{i-1,j-1} \vee s_{i-1,j} && \text{für } 1 < j \leq k \\ v_i &\Leftrightarrow x_i \wedge s_{i-1,k} \end{aligned}$$

Mit diesen Variablen können die folgenden Klauseln erstellt werden

$$\begin{aligned} &(\overline{x_1} \vee s_{1,1}) \\ &(\overline{s_{1,j}}) \quad \text{für } 1 < j \leq k \\ \\ &\left. \begin{aligned} &(\overline{x_i} \vee s_{i,1}) \\ &(\overline{s_{i-1,1}} \vee s_{i,1}) \\ &(\overline{x_i} \vee \overline{s_{i-1,j-1}} \vee s_{i,j}) \\ &(\overline{s_{i-1,j}} \vee s_{i,j}) \\ &(\overline{x_i} \vee \overline{s_{i-1,k}}) \end{aligned} \right\} \quad \text{für } 1 < j \leq k \quad \left. \vphantom{\begin{aligned} &(\overline{x_i} \vee s_{i,1}) \\ &(\overline{s_{i-1,1}} \vee s_{i,1}) \\ &(\overline{x_i} \vee \overline{s_{i-1,j-1}} \vee s_{i,j}) \\ &(\overline{s_{i-1,j}} \vee s_{i,j}) \\ &(\overline{x_i} \vee \overline{s_{i-1,k}}) \end{aligned}} \right\} \quad \text{für } 1 < i < n \\ \\ &(\overline{x_n} \vee \overline{s_{n-1,k}}) \end{aligned}$$

FA: Vernünftig links bündig machen

Diese Klauseln stellen sicher, dass genau k Variablen von $(x_1 \vee x_2 \vee \dots \vee x_n)$ wahr sind. Alle Literale, die auf $s_{i,j}$ basieren, müssen mit Hilfsvariablen verknüpft werden.

Bei den Codierungen besteht die Möglichkeit, die Kardinalität auf den exakten Sollgrad oder den mindestens Sollgrad zu setzen. Insgesamt erzielen beide Bedingungen denselben Effekt, da der maximale Grad durch die Anzahl der Kanten begrenzt ist. Wenn jedoch nur gefordert wird, dass der Knoten mindestens den Sollgrad erreicht, müssen weniger

3.1. Entscheidungsproblem

Hilfsvariablen eingeführt werden. Bei der exakten Methode könnte der Solver allerdings schneller zu einer Lösung finden, da diese Bedingung stärker ist, also mehr Informationen liefert.

Die andere Methode ist die sogenannte *subset* Methode. Bei dieser Methode wird für jeden Knoten i eine Menge von Klauseln erstellt. Diese Klauseln umfassen alle Kombinationen der Kanten E_i der Größe $|E_i| - (\deg^*(i) - 1)$. Wobei E_i die Menge aller Kanten für den Knoten i ist. Angenommen, der Knoten v_1 besitzt die Kanten e_1, e_2, e_3, e_4 und den Sollgrad 2, dann werden folgende Klauseln erstellt

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_4) \wedge (x_1 \vee x_3 \vee x_4) \wedge (x_2 \vee x_3 \vee x_4)$$

Damit diese vier Klauseln erfüllt sind, müssen mindestens zwei der Variablen wahr sein. Die Idee basiert darauf, dass für jede Klausel angenommen wird, dass nur noch eine Kante zum Erreichen des Sollgrads fehlt und die in der Klausel genannten Kanten noch nicht aktiv sind. Insgesamt wird so eine Kardinalitäts Codierung erzeugt. Diese Methode hat den Vorteil, dass keine Hilfsvariablen benötigt werden. Allerdings entstehen

$$\binom{|E_i|}{|E_i| - (\deg^*(i) - 1)}$$

viele Klauseln. Das wären in O-Notation

$$O\left(n \cdot \binom{|E_{\max}|}{|E_{\max}| - d_{\min}}\right)$$

viele Klauseln. Wobei $E_{\max}$ die maximale Kantenanzahl pro Knoten und $d_{\min}$ der minimale Sollgrad ist. Da dies nicht polynomial ist, werden zu viele Klauseln erzeugt.

Die Fixierung von Kanten kann ähnlich zur Mathematischen Formulierung umgesetzt werden. Hierbei werden anstatt die Variablen auf *Wahr* und *Falsch* zu setzen, die Klauseln (x_e) und $(\bar{x}_e)$ hinzugefügt.

Für die *alternative Kantenbeschränkung* können folgende Klauseln genutzt werden:

$$x_e \vee \bigvee_{e_j \in I(e)} x_{e_j} \quad \forall e \in E$$

3.1.2. Dreiecksansatz

Bei dem Dreiecksansatz wird nicht jeder Kante eine Variable zugeordnet. Stattdessen erhält jedes mögliche Dreieck eine eigene Variable.

Die Gradbeschränkung kann aus der Kantenformulierung übernommen werden. Lediglich die Knoten auf der Hülle müssen gesondert betrachtet werden. Für diese Knoten muss der Sollgrad um eins reduziert werden, da ein Dreieck am Rand zwei Kanten der Hülle stellt.

Für die Überschneidungsbeschränkung können zwei verschiedene Ansätze genutzt werden. Zum einem können sich überschneidende Dreiecke ausgeschlossen werden. Die andere Möglichkeit ist es, die Kanten zu betrachten. Wenn man die Kanten auf der Hülle

3. Lösungsansätze

gegen den Uhrzeigersinn betrachtet, muss auf der linken Seite immer genau ein Dreieck aktiv sein. Für alle anderen inneren Kanten müssen auf der linken sowie auf der rechten Seite immer genau gleich viele Dreiecke aktiv sein. Da es in einer Triangulierung keine Überschneidungen gibt, gibt es hierbei nur zwei Möglichkeiten. Entweder ist auf der linken Seite und auf der rechten Seite kein Dreieck aktiv oder auf beiden Seiten genau ein Dreieck aktiv.

Auch hier können Dreiecke ausgeschlossen werden. Dafür können die gleichen Bedingungen wie beim Ausschluss von Kanten verwendet werden. Es werden dann alle Dreiecke ausgeschlossen, die eine ausgeschlossene Kante enthalten.

Formulierungen

Für diese Formulierungen wird ebenfalls angenommen, dass V die Menge aller Knoten und E die Menge aller möglicher Kanten sei, also der Kanten die keine Punkte schneiden sei. Zusätzlich ist T die Menge leerer Dreiecke, da nur diese in einer Triangulierung vorkommen können. Für jedes Dreieck $t \in T$ wird eine Boolesche Variable x_t erstellt. Diese Variable ist wahr, wenn das Dreieck t in der Triangulierung enthalten ist. Die einzige nicht notwendige Bedingung die umgesetzt werden kann ist, das Ausschließen von Dreiecken. Dies kann analog zum Ausschließen von Kanten umgesetzt werden, in dem alle Dreiecke ausgeschlossen werden, die eine ausgeschlossene Kante enthalten.

Mathematische Formulierung

Die Gradbeschränkung kann analog zum Kantenansatz übernommen werden. Für die Knoten auf der Hülle muss der Sollgrad nur um eins reduziert werden. In Formeln wird das wie folgt definiert.

$$\begin{aligned} \sum x_{t_i} &= \deg^*(i) & \forall i \in V \setminus V_H \\ \sum x_{t_i} &= \deg^*(i) - 1 & \forall i \in V_H \end{aligned}$$

Wobei V_H die Menge der Knoten auf der Hülle ist und t_i alle Dreiecke für den Knoten i .

Der Ansatz ohne die Kantenüberschneidungen eignet sich besser für die mathematische Formulierung. Er kann mit folgenden Formeln umgesetzt werden.

$$\begin{aligned} \sum_{t \in T_e} x_t &= 1 & \forall e \in E_H \\ \sum_{t \in T_{e,l}} x_t &= \sum_{t' \in T_{e,r}} x_{t'} & \forall e \in E \setminus E_H \end{aligned}$$

Ebenfalls bezeichnet E_H die Menge der Kanten auf der Hülle und $T_{e,l}$ und $T_{e,r}$ die Menge aller Dreiecke auf der linken und rechten Seite der Kante e . Bei der Menge E_H ist wichtig zu beachten, dass die Kanten gegen den Uhrzeigersinn betrachtet werden.

KNF Formulierung

Für die Gradbeschränkung müssen auch hier die Sollgrade der Knoten auf der Hülle um eins reduziert werden. Diese Kardinalitäten können anschließend mit den Codierungen aus Abschnitt 3.1.1 umgesetzt werden. Für die Überschneidungen werden in diesem Fall sich überschneidende Dreiecke ausgeschlossen. Dies kann mit folgender Formel umgesetzt werden.

$$\overline{x_{t_1}} \vee \overline{x_{t_2}} \quad \forall t_1, t_2 \in T : t_2 \in I(t_1)$$

Der Ansatz ohne Kantenüberschneidungen eignet sich auf Grund der fehlenden direkten Kardinalitätsunterstützung nicht für die KNF Formulierung.

3.2. Optimierungsproblem

Im Gegensatz zu den vorherigen Ansätzen wird DCT in diesem Abschnitt nicht als Entscheidungsproblem, sondern als Optimierungsproblem betrachtet. Dies hat zum einen den Vorteil, dass bei einem *Timeout* des Solvers nicht der gesamte Fortschritt verloren geht. Ein weiterer Vorteil ist, dass bei Nein-Instanzen versucht wird, dennoch eine Lösung zu finden. Das bedeutet, sollte es in bestimmten Bereichen nicht möglich sein, eine DCT zu finden, kann zumindest eine möglichst nahe Triangulierung bestimmt werden.

Für die Bewertung der Lösungen wurde folgende Formel eingeführt. Diese Formel berechnet den Gesamtdurchschnitt von allen prozentualen Gradabweichungen.

$$\text{diff}_i = \left| \deg^*(i) - \deg(i) \right| \quad (3.1)$$

$$e_i = \begin{cases} \frac{\deg^*(i) - \text{diff}_i}{\deg^*(i)} & \deg^*(i) \geq \text{diff}_i \\ 0 & \text{sonst} \end{cases} \quad (3.2)$$

$$Q = \sum_{i=0}^n \frac{e_i}{n} \quad (3.3)$$

Diese Formel liefert eine Bewertung (Q) für eine Triangulierung mit n Knoten. In Formel (3.1) wird die Differenz zwischen dem gewünschten und aktuellen Grad berechnet. In Formel (3.2) wird diese Abweichung prozentual bewertet. Am Ende wird in Formel (3.3) ein Durchschnitt über alle prozentualen Werte gebildet.

Ansätze

Es werden drei Ansätze vorgestellt. Der erste ist ein naiver Ansatz der versucht mit Flips die Gradbeschränkung zu erreichen. Die anderen beiden Ansätze verwenden einen mathematischen Ansatz. Einen KNF Ansatz ist nicht möglich, da KNF Formel nur für Entscheidungsprobleme geeignet sind.

3. Lösungsansätze

Flips

In dieser Heuristik werden die in Definition 2.6 definierten Kantenflips verwendet. Ziel ist es, die Gradbeschränkung zu erreichen. Dabei wird ausgenutzt, dass Flips den Grad eines Knotens verändern können, die Triangulationseigenschaft dabei aber erhalten bleibt.

Das Verfahren beginnt mit einer initialen Triangulierung. Diese wird durch ein klassisches Triangulationsverfahren erzeugt. Anschließend werden maximal k Flips durchgeführt.

Zunächst wird mit Algorithmus 1 eine Kante ausgewählt. Diese soll geflippt werden. Der Algorithmus versucht, eine Kante zu finden, deren Knoten nicht den Sollgrad erfüllen. Zusätzlich wird über den Parameter p eine Wahrscheinlichkeit eingeführt. Dadurch kann auch eine Kante ausgewählt werden, deren Knoten den Sollgrad erfüllen. Dies verhindert, dass das Verfahren in einem lokalen Optimum stecken bleibt.

Die ausgewählte Kante e_0 wird anschließend wie folgt geflippt. Für diese Kante werden zunächst alle Dreiecke gesucht, die diese Kante enthalten. Dies geschieht, indem für die beiden Knoten $x_{0,0}$ und $x_{0,1}$ der Kante e_0 alle ausgehenden Kanten verglichen werden. Sollten zwei dieser Kanten den gleichen Knoten v haben, muss das Tripel $(x_{0,0}, x_{0,1}, v)$ ein Dreieck bilden, welches die Kante e_0 enthält. Sollten mehr als zwei Dreiecke gefunden werden, so müssen die beiden Dreiecke ausgewählt werden, welche leer sind (keine weiteren Knoten enthalten). Da es in einer Triangulierung nicht zu Überschneidungen kommen darf, kann es nur zwei leere Dreiecke geben. Aus diesen beiden Dreiecken $(x_{0,0}, x_{0,1}, v_0)$ und $(x_{0,0}, x_{0,1}, v_1)$ wird dann die Kante e_0 entfernt und durch die neue Kante, bestehend aus v_0 und v_1 , ersetzt.

Nach jedem Flip wird überprüft, ob die Gradbeschränkung für alle Knoten erfüllt ist. Sollte dies der Fall sein, wird das Verfahren beendet.

Algorithm 1 CHOOSE EDGE(v, p)

```
1:  $E \leftarrow \{e \mid e \text{ ist ausgehende Kante von } v\}$ 
2: while  $E \neq \emptyset$  do
3:    $e \leftarrow$  zufällige Kante aus  $E$ 
4:    $E \leftarrow E \setminus \{e\}$ 
5:    $u \leftarrow$  Knoten von  $e$  and  $v \neq u$ 
6:   if  $\deg(u) \neq \deg^*(u)$  then
7:     return  $e$ 
8:   else if zu  $p$  % then
9:     return  $e$ 
10:  end if
11: end while
12: return  $e$ 
```

Mathematische Ansatz

Für beide Ansätze können die Bedingung aus Abschnitt 3.1.1 übernommen werden. Nur die Gradbeschränkung wird durch eine Zielfunktion ersetzt. Zum einen wird als

3.3. Conflict-driven clause learning

Zielfunktion die Formel (3.3) genutzt. Für diese Zielfunktion wird eine angepasste Version der Formel (3.3) Maximiert. Die Teiler in (3.3) und (3.2) werden entfernt. Diese sind für die Maximierung nicht relevant.

Eine Fallunterscheidung für (3.2) ist für eine Zielfunktion nicht sinnvoll. Die Variablen e_i werden daher mit folgender Formel berechnet.

$$e_i = \deg^*(i) - \min(\text{diff}_i, \deg^*(i))$$

Zusammengefasst ergibt das folgende Zielfunktion.

$$\max \left(\sum_{i=0}^n \deg^*(i) - \min(\text{diff}_i, \deg^*(i)) \right)$$

Die andere Zielfunktion minimiert die maximale Gradabweichung. Dafür wird eine Hilfsvariable max_min eingeführt. Mit folgenden Formeln kann die Zielfunktion definiert werden

$$\begin{aligned} max_min &\geq \text{diff}_i & \forall i \in V \\ max_min &\geq 0 \end{aligned}$$

Die Variable max_min wird als Zielfunktion minimiert.

3.3. Conflict-driven clause learning

In diesem Kapitel werden Sat-Solver genauer betrachtet. SAT-Solver werden genutzt um KNF Formulierungen zu lösen. In diesem Abschnitt sollen keine weiteren Klauseln gefunden werden, sondern der Solver wird während des Lösens optimiert. Dafür wird *Conflict-driven clause learning* (CDCL) betrachtet. Die in diesem Kapitel vorgestellten Informationen stammen aus dem Buch *The Art of Computer Programming, Volume 4, Fascicle 6: Satisfiability* [14] Im Allgemeinen funktionieren SAT-Solver so, dass sie jede Möglichkeit ausprobieren. Dies lässt sich als Baum darstellen (siehe Abbildung 3.5). Natürlich werden Algorithmen angewendet, die diese Suche effizienter machen. Eine Möglichkeit die relevanten Äste effektiv zu untersuchen, ist der CDCL-Ansatz. Das Besondere am CDCL-Ansatz ist das es einen Pfad gibt, der eine aktuelle Teilbelegung der Literale speichert. Zusätzlich muss für jedes belegte Literal eine Begründung in Form einer Klausel angegeben werden. Diese Begründungen werden bei Konflikten relevant. Sollte es dazu kommen das der Solver keine weiteren Literale durch Begründungen setzen kann, wird ein Literal ohne Begründung gesetzt. Dieser Schritt wird als neue Entscheidungsebene bezeichnet.

3.3.1. Unit-Klauseln

Dieser Grundansatz zur Erweiterung des Pfades besteht darin, *Unit-Klauseln* zu finden. Das sind Klauseln, in denen nur ein Literal noch nicht belegt ist und die Klausel nicht

3. Lösungsansätze

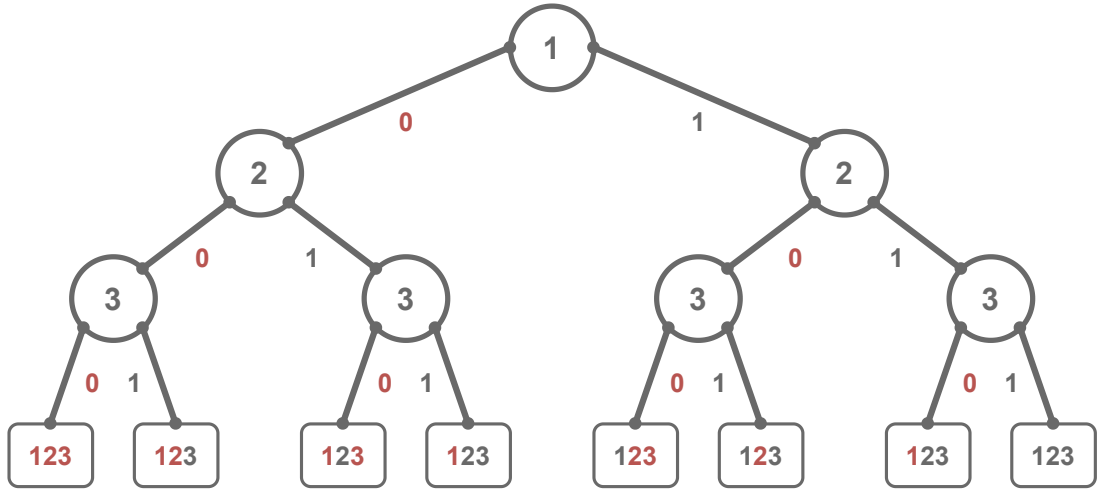


Abbildung 3.5.: Ein Baum welcher die Möglichen Entscheidungen eines SAT-Solvers darstellt. Jeder Ast entspricht einer Belegung der Literale. Die Blätter sind entweder erfüllende Belegungen oder Konflikte. Wobei eine rote Zahl für eine nicht belegtes Literal steht und eine schwarze Zahl für ein belegtes Literal.

erfüllt ist. Zum Beispiel kann durch die Klausel $(\overline{x_1} \vee x_2 \vee \overline{x_3})$ und dem Pfad $(\overline{x_2}, x_3)$ das Literal $\overline{x_1}$ hinzugefügt werden. Die Klausel dient dabei als Begründung.

Sollten keine weiteren *Unit-Klauseln* existieren, wird basierend auf Heuristiken ein Literal belegt. Bei dieser Art der Belegung wird intern vermerkt, dass eine neue Entscheidungsebene erreicht wurde. Die Entscheidungsebenen werden auch bei Konflikten wichtig. Es ist zu beachten, dass jeder Abschnitt des Pfades einer Entscheidungsebene zugeordnet werden kann. Ein Literal, das weiter rechts im Pfad steht, wird einer höheren Entscheidungsebene zugeordnet.

3.3.2. Konflikte

Ein Konflikt tritt auf, wenn ein Literal dem Pfad hinzugefügt werden soll, es aber bereits eine andere Belegung besitzt. Hier zeigt sich der besondere Ansatz von CDCL. Kommt es zu einem Konflikt, wird nicht einfach zu einer vorherigen Entscheidungsebene zurückgekehrt. Stattdessen werden die Informationen, wie es zu dem Konflikt kam, zur Lösung des Problems genutzt. Sei

$$c = (\overline{l} \vee \overline{a_1} \vee \dots \vee \overline{a_k})$$

eine Klausel, die einen Konflikt aufweist. Das bedeutet, alle Literale wurden dem Pfad hinzugefügt und die Klausel ist nicht erfüllt.

Es kann angenommen werden, dass l und ein weiteres Literal a_i aus c auf der höchsten Entscheidungsebene d liegen. Andernfalls wäre c eine *Unit-Klausel* und somit wäre l

3.3. Conflict-driven clause learning

bereits auf $d - 1$ belegt worden. Weiterhin kann angenommen werden, dass das Literal l das rechteste Literal von allen Literalen aus c im Pfad ist. Also das Literal aus c , das zuletzt belegt wurde. Daraus folgt, dass l keine neue Entscheidungsebene erzeugt hat, da zumindest a_i vor l auf der Entscheidungsebene d belegt wurde.

Es existiert somit eine Begründung

$$(\bar{l} \vee \bar{r}_1 \vee \dots \vee \bar{r}_k)$$

für die Belegung von l . Damit kann folgende Klausel erzeugt werden

$$c' = (\bar{a}_1 \vee \dots \vee \bar{a}_k \vee \bar{r}_1 \vee \dots \vee \bar{r}_k)$$

wobei c' mindestens eine Belegung aus der Entscheidungsebene d enthält. Sollte es ein weiteres Literal außer a_i geben, das auf der Entscheidungsebene d belegt wurde, ist c' ebenfalls eine Konfliktklausel. In diesem Fall wird c' rekursiv so lange erzeugt, bis es genau ein Literal $\bar{x}$ gibt, das auf Entscheidungsebene d belegt wurde. Der rekursive Aufruf fügt dabei die Begründung des zweiten Literals aus Entscheidungsebene d ohne dieses Literal zu c' hinzu. Von den restlichen Literalen in c' wird dann die höchste Entscheidungsebene bestimmt und zu dieser zurückgekehrt. Anschließend wird $\bar{x}$ dem aktuellen Pfad hinzugefügt und c' als Begründung für die Belegung verwendet.

4. Instanzgenerierung

In diesem Kapitel werden die verschiedenen Instanzentypen und ihre Generierung dargestellt. Auf diesen Instanzen werden die späteren Experimente ausgeführt.

Die Knotenmengen werden hierbei durch die von Python bereit gestellte Funktion *randint* generiert. Mit dieser werden so lange verschiedene Koordinaten berechnet, bis die Gewünschte Anzahl erreicht wurde.

4.1. Ja - Instanzen

Bei den Ja-Instanzen handelt es sich um Knotenmengen, bei denen die $\deg^*$ -Funktion Grade liefert, welche DCT ermöglichen. Hierbei ist nicht bekannt, wie viele mögliche Lösungen existieren, sondern nur, dass es mindestens eine gibt.

Der generelle Ablauf zur Erzeugung von Ja-Instanzen beginnt mit einer zufälligen Knotenmenge. Dabei wird lediglich die Anzahl der Knoten vorgegeben. Anschließend wird mit einem klassischen Triangulationsverfahren eine initiale Triangulierung erstellt. Von dieser werden dann die Grade gespeichert, um sie als Eingabe für die zu testenden Algorithmen zu verwenden.

Alle fünf Verfahren zur Erzeugung von Ja-Instanzen werden in Abbildung 4.1 visualisiert.

4.1.1. Delaunay

Das erste klassische Verfahren zur Gradbestimmung verwendet die Delaunay-Triangulierung. Die Delaunay-Triangulierung wurde bereits im Kapitel Abschnitt 1.3.1 vorgestellt. Für die Erstellung der Delaunay-Triangulierung wird die Bibliothek *scipy* genutzt.

4.1.2. Delaunay mit zufälligen Flips

Das zweite Verfahren zur Erzeugung von Ja-Instanzen erweitert das Delaunay Verfahren, indem zufällige Flips durchgeführt werden. Zunächst wird eine Delaunay-Triangulierung der Knotenmenge erstellt. Auf dieser Triangulierung werden anschließend k zufällige Flips gemäß Definition 2.6 durchgeführt. Das genaue verfahren wurde in Abschnitt 3.2 schon beschrieben. Dieser Prozess verändert die lokale Struktur der Triangulierung und erzeugt so Knotenmengen mit unterschiedlichen Gradverteilungen, behält aber die Triangulationseigenschaft bei.

4. Instanzgenerierung

4.1.3. Zufällige Kantenerzeugung

Das nächste Verfahren basiert auf der zufälligen Auswahl von Kanten. Hierbei werden zunächst alle möglichen Kanten zwischen den Punkten identifiziert. In ersten Schritt werden Kanten ausgeschlossen, welche andere Knoten schneiden würden.

Die verbleibenden gültigen Kanten werden dann in einer zufälligen Reihenfolge betrachtet. Für die betrachteten Kanten wird geprüft ob sie eine der bereits ausgewählten Kanten schneidet. Nur wenn keine Schnittkanten entstehen, wird die Kante der Triangulierung hinzugefügt.

Dieses Verfahren ist maximal da alle möglichen Kanten iterative betrachtet wurden.

4.1.4. Iteration

Ein weiteres Verfahren zur Erzeugung von Ja-Instanzen basiert auf der Iteration. Hierbei werden zunächst alle möglichen Kanten zwischen den Punkten identifiziert. Diese werden nun nacheinander betrachtet und wenn sie keine andere Kante schneiden hinzugefügt. Diese Verfahren ist ebenfalls maximal, da alle möglichen Kanten betrachtet wurden. Beim Iterativen Verfahren ist zu beachten das Triangulationen erzeugt werden, welche einen Knoten haben welcher einen sehr hohen Sollgrad hat. Dies basiert darauf, dass die Kanten in der Reihenfolge betrachtet werden wie sie erstellt werden. Das hat den Effekt, dass alle Kanten des ersten Knoten hinzugefügt werden können, da es noch keine anderen Kanten gibt welche diese schneiden könnten.

4.1.5. Greedy

Das letzte Verfahren ist das Greedy Verfahren. Hierbei werden auch alle möglichen Kanten zwischen den Punkten identifiziert. Allerdings werden sie im Gegensatz zum iterativen Verfahren zunächst nach der Länge aufsteigend sortiert. Anschließend wird wie beim Iterativen Verfahren jede mögliche Kante nacheinander hinzugefügt.

4.2. Nein-Instanzen

Nein-Instanzen sind Knotenmengen, bei denen die $\deg^*$ Funktion Gradwerte liefert, die DCT nicht ermöglichen. Diese Instanzen entstehen nicht direkt, sondern werden aus Ja Instanzen abgeleitet. Nach der Generierung muss zusätzlich geprüft werden, ob die Instanz nicht möglich ist. Da die beschriebenen Verfahren nur mit hoher Wahrscheinlichkeit Nein Instanzen erzeugen. Es ist zu beachten, dass die Summe der Sollgrade mithilfe der Polyederformel einfach berechnet und überprüft werden kann. Daher besitzen die abgeleiteten Nein Instanzen insgesamt die gleichen Sollgrade wie die Ja-Instanzen. Zusätzlich darf kein Sollgrad größer als die Anzahl der Knoten abzüglich eins sein, da sonst der Sollgrad nicht erfüllt werden kann, da nicht genügend Kanten erzeugt werden können.

Bei der ersten Methode werden zwei Knoten ausgewählt. Vom ersten Knoten wird ein Wert abgezogen und beim zweiten Knoten derselbe Wert hinzugefügt. Zunächst wird eine

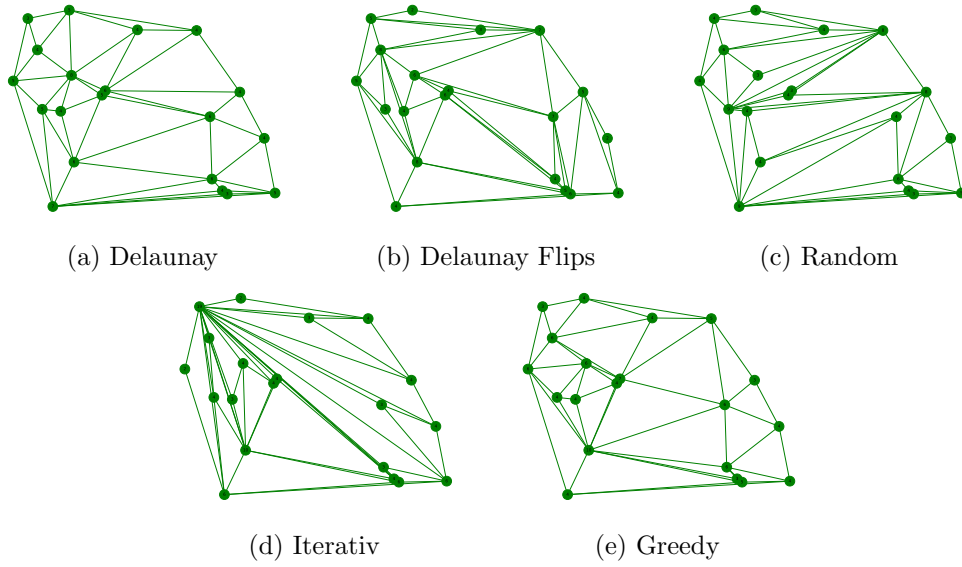


Abbildung 4.1.: Visualisierung der fünf verschiedenen Instanztypen.

zufällige Zahl r zwischen null und c gewählt. Hierbei ist c eine konstante Zahl, die beim Generieren festgelegt wird. Anschließend werden zwei Knoten v_1 und v_2 ausgewählt. Dabei müssen folgende Bedingungen erfüllt sein

$$v_1 \neq v_2 \quad (4.1)$$

$$\deg^*(v_1) < r + 2 \quad (4.2)$$

$$\deg^*(v_2) + r > n - 1 \quad (4.3)$$

wobei n die Anzahl der Knoten ist. Wenn beide Knoten die Bedingungen erfüllen, wird der Sollgrad von v_1 um r verringert. Der Sollgrad von v_2 wird um r erhöht.

Die zweite Methode besteht darin, zwei Knoten auszuwählen und deren Sollgrade zu tauschen. Es muss lediglich beachtet werden, dass es sich um zwei verschiedene Knoten handelt.

Beide Ansätze werden mehrfach angewendet, um die Wahrscheinlichkeit zu erhöhen, dass die Instanzen nicht mehr möglich sind. Knoten werden dabei allerdings nicht mehrfach betrachtet.

Die erste Methode wird im Folgenden *move* genannte. Die andere Methode wird *change* genannt.

4.3. Theoretische Mehrere Lösungen

Bei Ja-Instanzen stellt sich die Frage, ob sie eindeutig sind. Das bedeutet, ob es pro Ja-Instanz nur eine Lösung gibt und ob die Instanz nicht mehr erfüllbar ist, sobald eine Kante dieser Lösung verboten wird. Die Antwort darauf ist nein, es existieren Instanzen,

4. Instanzgenerierung

bei denen mehrere Lösungen möglich sind. Wie in Abbildung 4.2 zu erkennen ist, existieren

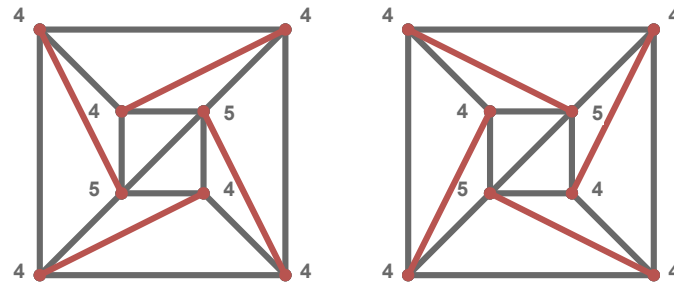


Abbildung 4.2.: Eine Ja-Instanz mit zwei möglichen Lösungen. Die Zahlen an den Knoten geben hierbei den Sollgrad an. Die roten kanten sind jene die in den Lösungen unterschiedlich sind.

zwei verschiedene Lösungen für die gleiche Ja-Instanz. Die Zahlen an den Knoten geben hierbei den Sollgrad an. Diese Eigenschaft wird auch nochmal in Abschnitt 6.12.1 im Zusammenhang mit der Schwierigkeit einer Instanz untersucht. Das es mehrere Lösungen gibt lässt sich darauf zurück führen das der Sollgrad hier *im* und *gegen* den Uhrzeigersinn erfüllt wird. Diese Art von Konstruktion kann theoretisch unendlich fortgesetzt werden, diese Eigenschaft könnte genutzt werden auf andere Probleme zu reduzieren. Also die Anzahl der Lösungen einer Instanz kann als eine Eigenschaft dieser betrachtet werden.

5. Solver

In dieser Arbeit werden drei verschiedene Solver verwendet PySAT, OrTools und Gurobi. In diesem Kapitel wird eine Übersicht über diese gegeben und eine Implementierung der in Kapitel 3 beschriebenen Ansätze.

5.1. Generelle Implementierungen

Generelle Implementierungen welche in alle Solvern verwendet werden. Zum einen wird betrachtet wie der Graph verwaltet wird. Danach wird erklärt wie die Schnittkanten bestimmt werden. Im letzten Abschnitt wird eine Implementierung für das in Abschnitt 5.1 beschriebene Ausschließen von Kanten anhand von Schnittkanten erläutert.

Graph operationen

Alle Kanten werden mit der Bibliothek *Networkx* verwaltet. Mit dieser können auch die inzidente Kanten eines Knotens abgefragt werden. Beim Erstellen der Kanten muss getestet werden, ob die Kanten keinen Knoten schneiden. Dies wird mit der Bibliothek *Shapely* getestet. Bei der Berechnung der Hülle muss beachtet werden, dass alle Knoten, welche auf der Hülle liegen betrachtet werden. Die *convex_hull* Funktion von Shapely liefert nur die Eckpunkte der Hülle. Damit auch Punkte auf einer Geraden zwischen zwei Eckpunkten betrachtet werden, muss nach der Berechnung der Convexen Hülle noch der Schnitt zwischen Convexer Hülle und allen Punkten berechnet werden. In diesem Schnitt sind dann alle Punkte enthalten, welche auf der Hülle liegen.

Schnittkanten

Für die Bestimmung der Schnittkanten wird ein Algorithmus verwendet, der auf dem Konzept des *Rechtsknicks* beziehungsweise *Linksknicks* basiert. Dafür ist es notwendig, dass ein vollständiger Graph betrachtet wird, da angenommen wird, dass zwischen allen Knoten eine Kante existiert. Es wird jede Kante $e_i \in E$, bestehend aus den Knoten $v_{i,1}$ und $v_{i,2}$, nacheinander betrachtet. Anschließend werden zwei Knoten v_3 und v_4 gesucht, die auf unterschiedlichen Seiten der Kante liegen. Dabei gilt: Das Tripel $(v_{i,1}, v_{i,2}, v_3)$ bildet einen Rechtsknick und das Tripel $(v_{i,1}, v_{i,2}, v_4)$ einen Linksknick. Danach muss überprüft werden, ob $v_{i,1}$ und $v_{i,2}$ auf verschiedenen Seiten der Kante liegen, die durch v_3 und v_4 gebildet wird. Sind alle diese Bedingungen erfüllt, so ist die Kante (v_3, v_4) eine Schnittkante von e_i . Der Vorteil dieses Algorithmus liegt darin, dass alle vier Knicküberprüfungen notwendige Eigenschaften darstellen. In der Praxis werden dadurch viele Knoten bereits ausgeschlossen, da sie die erste Knickbedingung nicht erfüllen.

5. Solver

Schnittkanten ausschließen

Um die Kanten zu finden, bei denen die Schnittkanten zu viele Kanten ausschließen, wird jede Kante iterativ betrachtet. Für jede betrachtete Kante werden zunächst Zähler für jeden Knoten erstellt. Danach werden nacheinander alle Schnittkanten der betrachteten Kante aufgerufen. Für alle Schnittkanten wird der Zähler der beiden Endknoten der Schnittkanten um eins erhöht. Am Ende werden alle Knoten überprüft. Sollte ein Knoten v folgende Formel erfüllen wird die Kante ausgeschlossen.

$$\text{deg}(v) - \text{node_count}[v] > \text{deg}^*(v)$$

5.2. PySAT

PySAT ist eine Python-Bibliothek, die eine Sammlung von SAT-Solvern bereitstellt. Diese SAT-Solver entscheiden, ob eine KNF-Formel erfüllbar ist. Dementsprechend kann PySAT nur Entscheidungsprobleme, aber keine Optimierungsprobleme lösen. Für alle SAT-Solver wird eine einheitliche Schnittstelle in Python bereitgestellt. Zusätzlich werden verschiedene Codierungen für Kardinalitäten angeboten. Das bedeutet, man gibt für eine Menge von Variablen an, wie viele von ihnen wahr sein sollen und erhält eine KNF-Formel, die diese Bedingung abbildet. PySAT stellt hierfür verschiedene Codierungen mit Implementierung bereit. PySAT ist open source und kostenlos.

Für PySAT wird die KNF-Formulierung verwendet. Es wurde alle Bedingungen aus Abschnitt 3.1.1 und Abschnitt 3.1.2 umgesetzt.

5.3. OrTools

Der zweite Solver ist OrTools (Google Optimization Tools) von Google. Dieser unterstützt verschiedene Modellierungsarten wie lineare Programmierung (LP), gemischt-ganzzahlige Programmierung (MIP), Constraint Programming (CP) und Routing-Algorithmen. OrTools ist in C++ implementiert und es wird eine Python-Schnittstelle bereitgestellt. Mit OrTools können sowohl Entscheidungsprobleme als auch Optimierungsprobleme gelöst werden. Kardinalitätsbeschränkungen und andere lineare Bedingungen werden direkt unterstützt und müssen nicht codiert werden. OrTools ist open source und kostenlos unter der Apache License 2.0.

Für OrTools wird die mathematische Formulierung verwendet. OrTools würde auch die KNF-Formulierung unterstützen, allerdings ist die mathematische Formulierung durch die direkte Unterstützung von Kardinalitäten deutlich einfacher umzusetzen. Die Dreiecksformulierungen aus Abschnitt 3.1.2 können mit *add* direkt übergeben werden.

Die Kantenformulierung aus Abschnitt 3.1.1 kann ebenfalls übergeben werden. Allerdings können bei den Überschneidungen die Paare auch anders behandelt werden. So kann für jedes Paar die Funktion $\text{AddAtMostOne}(x_{e_1}, x_{e_2})$ oder $\text{AddBoolOr}(x_{e_1}.Not(), x_{e_2}.Not())$ verwendet werden.

FA: gucken das hier die Funktion nicht im Absatz unterbrochen werden

Für OrTools werden zusätzlich die Zielfunktionen aus Abschnitt 3.2 implementiert. Für die Durchschnitts-Zielfunktionen ist folgende Formel gegeben.

$$\text{diff}_i = \left| \deg^*(i) - \deg(i) \right| \quad (5.1)$$

$$\max \left(\sum_{i \in V} \deg^*(i) - \min(\text{diff}_i, \deg^*(i)) \right) \quad (5.2)$$

Da OrTools keine Betragsfunktion in Formeln erlaubt, muss die Funktion *AddAbsEquality* verwendet werden. Diese erzeugt den Betrag mit den folgenden Bedingung

$$\begin{aligned} \text{diff}_i &\geq \deg^*(i) - \deg(i) \\ \text{diff}_i &\geq \deg(i) - \deg^*(i) \\ \text{diff}_i &= (\deg^*(i) - \deg(i)) \vee (\deg(i) - \deg^*(i)) \end{aligned}$$

Die Variable diff_i ist hierbei eine Integer-Hilfsvariable, die den Betrag der Differenz erhält. Diese Variable kann dann für die Formel (5.1) genutzt werden.

OrTools erlaubt zusätzlich keine *min*-Funktion in Formeln. Deshalb wird die Funktion *AddMinEquality* verwendet. Diese nutzt das eine Minimierung mit Folgender Formel in ein Maximierung umgewandelt werden kann.

$$\min(a, b) = -\max(-a, -b)$$

Eine Betragsfunktions kann auf eine Maximierungsfunktions reduziert werden. Da dies im Beispiel oben der Fall ist, können hier die gleichen angepassten Bedingungen genutzt werden.

Alle angepassten Formeln zusammengefasst sehen wie folgt aus:

$$\begin{aligned} \text{diff}_i &\geq \deg^*(i) - \deg(i) \\ \text{diff}_i &\geq \deg(i) - \deg^*(i) \\ \text{diff}_i &= (\deg^*(i) - \deg(i)) \vee (\deg(i) - \deg^*(i)) \\ \text{min}_i &\geq -\text{diff}_i \\ \text{min}_i &\geq -\deg^*(i) \\ \text{min}_i &= (-\text{diff}_i) \vee (-\deg^*(i)) \\ e_i &= \deg^*(i) - (-\text{min}_i) \\ Q_T &= \sum e_i \end{aligned}$$

Als Zielfunktion wird dann Q_T maximiert.

5.4. Gurobi

Der letzte Solver ist Gurobi. Es werden ebenfalls Modellierungen wie LP, MIP und quadratische Programmierung (QP) unterstützt. Gurobi ist in c implementiert und bietet

5. Solver

eine Python-Schnittstelle an. Gurobi ist kostenpflichtig, bietet jedoch eine kostenlose Lizenz für die Forschung an.

Für Gurobi kann die mathematische Formulierung genutzt werden. Umgesetzt wurde zum einen der Kantenansatz aus Abschnitt 3.1.1 und der Dreiecksansatz aus Abschnitt 3.1.2. Gurobi bietet durch die *addConstr* Funktion direkt die Möglichkeit alle Formeln hinzuzufügen.

5.5. Simplified CDCL Solve

Der CDCL Solver namens *Simplified CDCL Solve (CaDiCaL)* [4] von Armin Biere ermöglicht es, eigene Funktionen in den Lösungsprozess einzubinden. Diese Funktion erlaubt es Literale mit Begründung zu setzen. Im Folgenden wird diese Funktion *Propagation* genannt. Die Idee bei dieser *Propagation* ist es, die Bedingungen aus Abschnitt 3.1.1 auf Teilinstanzen anzuwenden.

Es wird angenommen, dass die Hülle fixiert wurde und alle Kanten, die die gesamte Instanz halbieren, bereits getestet wurden. Der Grund hierfür ist, dass die Hülle eine geeignete Fläche bietet, mit der alle Punkte betrachtet werden können.

Zur Umsetzung wird mit Hilfe eines Arrangements verfolgt, welche Kanten aktuell aktiv sind. Es ist darauf zu achten, dass bei jeder neuen Entscheidungsebene das Arrangement gespeichert und eindeutig einer Ebene zugeordnet wird. Tritt ein Konflikt auf, wird das Arrangement auf die entsprechende Entscheidungsebene zurückgesetzt.

Wird die *Propagation* aufgerufen, können alle Flächen des Arrangements betrachtet werden. Relevant sind dabei nur Flächen, die mehr als drei Kanten besitzen. Zusätzlich kann die unbegrenzte Fläche ausgeschlossen werden, da alle Punkte durch die Hülle eingeschlossen sind. Für diese Flächen können dann alle teilenden Kanten erzeugt werden, indem sämtliche Zweierkombinationen der Randknoten betrachtet werden. Dadurch wird die Fläche in zwei Hälften geteilt und die Kante kann getestet werden.

Vor dem Testen können alle Kanten übersprungen werden, die bereits festgelegt wurden oder keine gültigen Kanten sind. Für die Punktzuordnung kann zu Beginn jeder *Propagation* jeder Punkt einer Fläche zugeordnet werden. Sollte dann eine Kante getestet werden, können alle Punkte der zugehörigen Fläche betrachtet und ähnlich wie in Abschnitt 5.1 mit Hilfe von *Rechtsknicks* und *Linksknicks* einer Teilhälfte zugeordnet werden. So können alle Kanten der Hülle der Teilhälfte betrachten werden und der Punkt der Hälfte zugewiesen werden bei der nur *Rechtsknicks* oder nur *Linksknicks* vorhanden sind.

Führt das Testen einer Kante zu einem Konflikt, kann das zugehörige Literal l auf *falsch* gesetzt werden. Als Begründung kann folgende Klausel verwendet werden

$$(\bar{l} \vee \bigvee_{e \in E_A} \bar{x}_e)$$

wobei E_A die Menge der aktiven Kanten bezeichnet.

6. Auswertung

In diesem Kapitel werden die Ansätze aus Kapitel 3 ausgewertet. Die ersten beiden Experimente testen, ob die Solver bei gleichen Bedingungen die gleichen Ergebnisse liefern. Die beiden folgenden Experimente werden genutzt, um den besten SAT-Solver für DCT aus Abschnitt 5.2 zu finden. Danach folgt in Abschnitt 6.6 ein Experiment, welches die verschiedenen Solver mit den verschiedenen Beschränkungen vergleicht, um den besten Solver zu bestimmen. Dieser wird genutzt, um in den folgenden Experimenten zum einen die theoretische Ausschließung von Kanten zu testen, möglichst große Instanzen zu lösen und Ja-Instanzen bzw. Nein-Instanzen getrennt voneinander zu betrachten. Anschließend werden in Abschnitt 6.10 die Zielfunktionen betrachtet. In dem darauf folgenden Experiment wird der CDCL-Ansatz aus Abschnitt 5.5 genauer betrachtet, um während des Lösens Einfluss auf den Solver zu nehmen. Zum Abschluss wird in drei verschiedenen Experimenten versucht, die Schwierigkeit der Instanzen zu bestimmen.

6.1. Versuchsumgebung

In diesem Abschnitt werden die Bedingungen für die Experimente beschrieben. Alle Experimente welche eine Laufzeit beinhalten wurden entweder auf *AMD Ryzen 7 5800X @ 8x 3.8GHz-4.7GHz* mit *128GB* Arbeitsspeicher oder auf *AMD Ryzen 9 7900 @ 12x 3.7GHz-5.4GHz* mit *96GB* Arbeitsspeicher durchgeführt. Im Folgenden werden in den Experiment zwischen *Ryzen 7* und *9* unterschieden. Gespeichert werden die Ergebnisse mit der Python-Bibliothek *Algbench*. Dabei wurden für alle Experimente mit Ausnahme von Abschnitt 6.8 ein Timeout von 5 Minuten (300 Sekunden) gesetzt. Diese komplette Laufzeit also inklusive Vorbereitungszeit musste in der Timeoutzeit liegen.

Für die Instanzgenerierung wurden die in Kapitel 4 beschriebenen Methoden verwendet. Es wurden Instanzen mit 30 bis 200 Knoten in 10er Schritten generiert. Wobei immer zwei Ja-Instanzen und zwei Nein-Instanzen pro Knotengröße generiert wurden. Bei den Nein-Instanzen wurde jeweils eine *move* und eine *change* Instanz generiert. Im folgenden werden alle Instanzen nach der Knotengröße aufsteigend sortiert.

Alle Experimente wurden in Python 3.13.5 durchgeführt. Der Algorithmus zum Finden von Überschneidungen wurde in c++ implementiert und mit *pybind11* in Python eingebunden. Ebenso wurde der CDCL-Ansatz in c++ implementiert.

In vielen Experimenten werden sogenannte Kaktus-Diagramme verwendet. Bei diesen wird auf der X-Achse die Zeit und auf der Y-Achse die Anzahl der gelösten Instanzen angezeigt. Bei diesen Diagrammen werden alle Solver mit allen Instanzen *gleichzeitig* gestartet. Sollte ein Solver dann eine Instanz lösen, wird die Linie vertikal um eine Einheit nach oben versetzt. So ergibt sich eine Treppenlinie. Bei diesen Diagrammen

6. Auswertung

zeigt die Höhe die Anzahl der gelösten Instanzen an. Analog die Breite die benötigte Zeit. Dementsprechend ist es Positiv wenn eine Linie höher und weiter links ist. Wenn im folgenden keine Kaktus-Diagramme verwendet werden, werden die genutzen Diagramme genauer erklärt.

Zur Verbesserung der Lesbarkeit, sind einige Diagramme nur im Anhang enthalten. Diese sind am Buchstaben im Namen zu erkennen.

6.2. Permutation

FA: Diagramme alt anpassen

In diesem Experiment wird untersucht, ob es einen Einfluss hat, in welcher Reihenfolge die Knoten den Solvern übergeben werden. Es wurden Instanzen mit 30 bis 50 Knoten betrachtet und auf dem Ryzen 7 getestet. Als Beschränkungen kamen die Überschneidungsbeschränkung und die Gradbeschränkung zum Einsatz. Wie in Abbildung A.1, Abbildung A.2 und Abbildung A.3 zu erkennen ist, unterscheiden sich die Laufzeiten bei jedem Solver in mindestens einer Instanz. Daraus folgt, dass die Permutation der Knotenmenge einen Einfluss auf die Laufzeit hat. In den folgenden Experimenten wird für jeden Lauf die Knotenmenge fünfmal in unterschiedlichen Permutationen übergeben und der Durchschnitt berechnet.

6.3. Mehrmalige Durchläufe

FA: Diagramme alt anpassen

In diesem Experiment wird untersucht, ob sich bei mehrfachen Durchläufen mit den selben Instanz, mit dem selben Solver und gleichen Parametern die Laufzeit verändert. Auch hier wurden Instanzen mit 30 bis 50 Knoten betrachtet und auf dem Ryzen 7 getestet. Als Beschränkungen kamen die Überschneidungsbeschränkung und die Gradbeschränkung zum Einsatz. Gurobi und SAT hatten bei allen Instanzen die gleichen Laufzeiten. Bei OrTools hingegen variieren die Laufzeiten bei den meisten Durchläufen. In Abbildung B.1 und Abbildung B.3 ist zu erkennen, dass bei PySAT und Gurobi alle Durchläufe die gleichen Laufzeiten aufweisen. Nur bei OrTools ist in Abbildung B.2 zu sehen, dass die Laufzeiten unterschiedlich sind. Da in Abschnitt 6.2 bereits mehrere Durchläufe mit unterschiedlichen Permutationen durchgeführt wurden, werden die OrTools-Durchläufe nicht zusätzlich mehrfach ausgeführt.

6.4. SAT Gradbeschränkung

In diesem Experiment werden verschiedene Methoden der Gradbeschränkung in einem SAT Solver verglichen. Hierbei wurden für alle drei Methoden, also *subset*, Kardinalität mit Exaktem-Sollgrad und Kardinalität mit Mindest-Sollgrad, die Laufzeiten gemessen. Bei den Kardinalitäts Codierungen von PySat wurden auch alle möglichen Codierungen

getestet. Das Ziel dieser Optimierung ist es die Gradbeschränkung mit der besten Laufzeit zu finden, welche dann in den folgenden Experimenten verwendet wird. Für die Knotengröße wurden Instanzen der Größe 30 bis 90 verwendet. Bei der *subset* Methode wurde nur die Instanz mit 30 Knoten verwendet, da bei größeren Knotenmengen der zur Verfügung stehende Arbeitsspeicher zu klein war. Als zusätzliche Bedingung wurde die Überschneidungsbeschränkung verwendet.

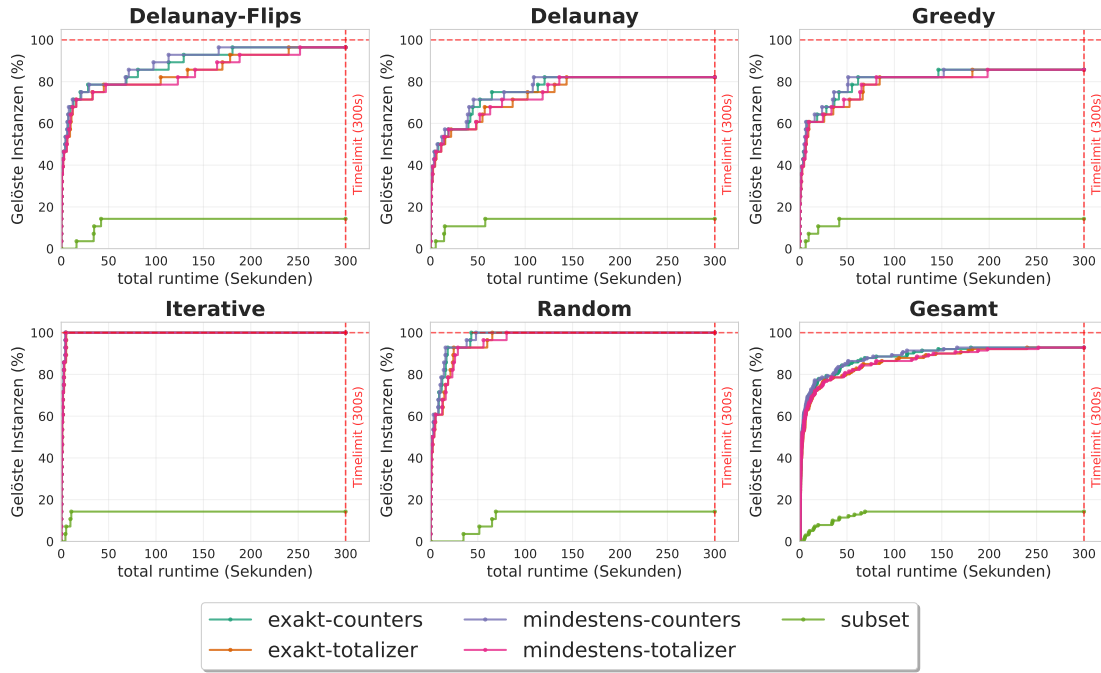


Abbildung 6.1.: In der Abbildung werden für alle Instanzen Arten die drei Kardinalitätsbeschränkungen verglichen. Dabei werden für die Exakte-Methode und die Mindest-Methode *sequential counters* und *totalizer* genutzt. Die *subset* Methode ist die schlechteste, da sie nur für kleine Instanzen funktioniert. Es zeigt sich, dass die Exakte-Methode sowie die Mindest-Methode bei *sequential counters* und *totalizer* sehr ähnliche Ergebnisse erzielen. *Sequential counters* ist dabei eindeutig schneller als *totalizer*. Im Gesamtüberblick lässt sich erkennen, dass die Mindest-Methode mit *sequential counters* minimal schneller ist als die Exakte-Methode mit *sequential counters*.

In Abbildung C.1 ist zu erkennen, dass *sequential counters* und *totalizer* bei der Exakten-Methode sowie bei der Mindest-Methode die besten Ergebnisse erzielen. Diese werden in Abbildung 6.1 noch mal verglichen. Hier ist zu sehen, dass *sequential counters* die besten Ergebnisse erzielt. Da die Exakte-Methode leicht besser ist, wird dieser in den folgenden Experimenten verwendet.

6.5. SAT Solver Vergleich

In diesem Experiment werden die verschiedenen SAT Solver verglichen, die von PySat bereitgestellt werden. PySat stellt *Cadical195*, *Gluecard4*, *Glucose42*, *Lingeling*, *MapleCh-*

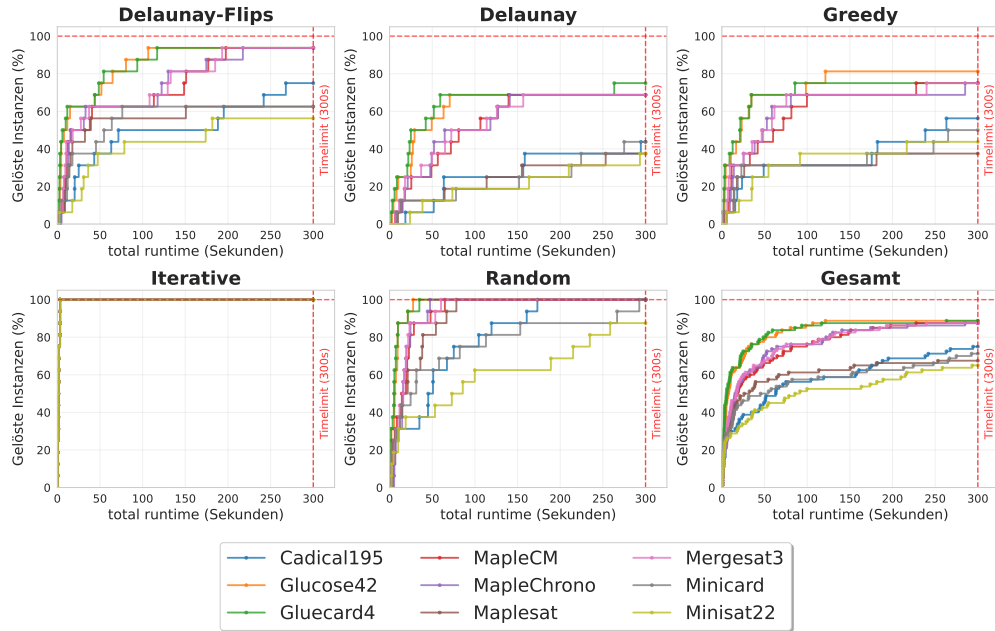


Abbildung 6.2.: In dieser Abbildung sind die Laufzeiten der verschiedenen SAT-Solver aus PySAT zu sehen. Hierbei ist klar erkennbar das *Gluecard4* und *Glucose42* die besten Ergebnisse erzielen. Dass diese beiden Solver ähnliche Ergebnisse liefern ist nicht verwunderlich, da sie beide auf dem Glucose-Algorithmus basieren. Es zeigt sich aber, dass bei diesem Problem weder der native noch der *sequential counters* Ansatz einen signifikanten Unterschied machen. Es ist aber zu erkennen, dass *Glucose42* bei Greedy eine Instanze in guter Zeit geschafft hat welche *Gluecard4* nicht lösen konnte.

rono, *MapleCM*, *Maplesat*, *Mergesat3*, *Minicard* und *Minisat22* zur Verfügung. Von diesen wurden jeweils die neuesten verfügbaren Versionen verwendet. Alle Solver nutzen die *sequential counters* Codierung sowie die Überschneidungsbeschränkung. Bei *Gluecard4* und *Minicard* kam anstelle von *sequential counters* eine native Codierung der Gradbeschränkung zum Einsatz. Diese wird nur für diese beiden Solver angeboten und bietet eine direkte Codierung ohne zusätzliche Variablen. Der *Lingeling* Solver wurde in großen Experimenten nicht berücksichtigt, da er kein Timeout unterstützt. Er wurde aber zusätzlich mit dem besten Solver verglichen und hat schlechtere Ergebnisse erzielt. *CryptoMinisat* wurde nicht verwendet, da dieser Solver speziell für XOR-Probleme optimiert ist. Für die Instanzengröße wurden 60 bis 90 Knoten gewählt und es wurde auf dem Ryzen 9 getestet. Das Ziel besteht darin, den Solver zu identifizieren, der die besten Ergebnisse liefert, um diesen in zukünftigen Experimenten einzusetzen.

In Abbildung 6.2 ist zu erkennen, dass *Gluecard4* und *Glucose42* die besten Ergebnisse erzielen. Für den weiteren Verlauf wird *Glucose42* verwendet, da dieser bei Greedy etwas besser war.

6.6. Vergleich der Modellierungstechniken

In diesem Experiment werden alle möglichen Bedingungen an die Solver übergeben und einzeln getestet. Untersucht werden die Solver PySAT, OrTools und Gurobi jeweils mit dem Kantenansatz sowie dem Dreiecksansatz. Für alle Durchläufe wurden die Überschneidungsbeschränkung und die Gradbeschränkung genutzt. Bei dem Kantenansatz wurden das Fixieren und Ausschließen von Kanten sowie die *alternative Kantenformulierung* getestet. Bei dem Dreiecksansatz wurde nur das Ausschließen von Kanten getestet. Die Instanzengröße liegt bei 30 bis 120 Knoten und es wurde auf dem Ryzen 7 getestet.

In Abbildung D.4 und Abbildung D.2 ist zu erkennen, dass Gurobi und PySAT am besten mit dem fixieren der Hülle optimiert werden. Bei OrTools hingegen erzielt die alternative Kantenformulierung die besten Ergebnisse, wie in Abbildung D.3 zu sehen ist.

In Abbildung 6.3 ist zu sehen das OrTools die besten Ergebnisse erzielt, gefolgt von PySAT. In folgenden Experimenten wird OrTools zusammen mit der alternativen Kantenformulierung und der fixierung der Hülle verwendet.

FA: Dreiecke sind noch nicht final da gen3 blockiert ist

FA: Kanten fixieren hatte einene Fehler und muss neu getestet werden. Sollte aber keine Relevanz haben und am Text nichts ändern.

6.7. Kanten vorab berechnen

In Abschnitt 6.6 wurde festgestellt, dass das Ausschließen und Fixieren von Kanten nur schlecht Ergebnis liefert. Die Hypothese dazu ist, dass zu wenig Kanten ausgeschlossen bzw. fixiert wurden. Um das zu überprüfen wird vor dem Experiment eine Menge von Kanten bestimmt, die sicher ausgeschlossen oder fixiert werden können. Somit können während des Lösungsprozess beliebig viele Kanten ausgeschlossen oder fixiert werden. In diesem Experiment wurde OrTools mit der Überschneidungsbeschränkung und Gradbeschränkung ausgewählt. Für die Knotengröße wurden Instanzen mit 30 bis 80 Knoten gewählt und das Experiment wurden auf dem Ryzen 9 durchgeführt. Um das Experiment zu vereinfachen wurden nur Instanzen betrachtet, die genau eine Lösung besitzen. Somit konnte eine beliebige Lösung der Instanze genommen und mit dieser die Kanten bestimmt werden, welche fixiert oder ausgeschlossen werden soll. Anhand von Abbildung I.1 und Abbildung 6.4 lässt sich bestätigt, dass das Ausschließen und Fixieren der Kanten in Abschnitt 6.6 schlecht war, weil zu wenig Kanten ausgeschlossen bzw. fixiert wurden. Es ist überraschend, dass es beim Fixieren keinen unterschied macht, ob 50 oder 80 Prozent der Kanten fixiert wurden. Dass dies beim Ausschließen nicht das Fall ist, lässt sich damit erklären, dass zum einem mehr Kanten theoretisch ausgeschlossen werden können und somit der absolute Unterschied größer ist. Zum andern,

6. Auswertung

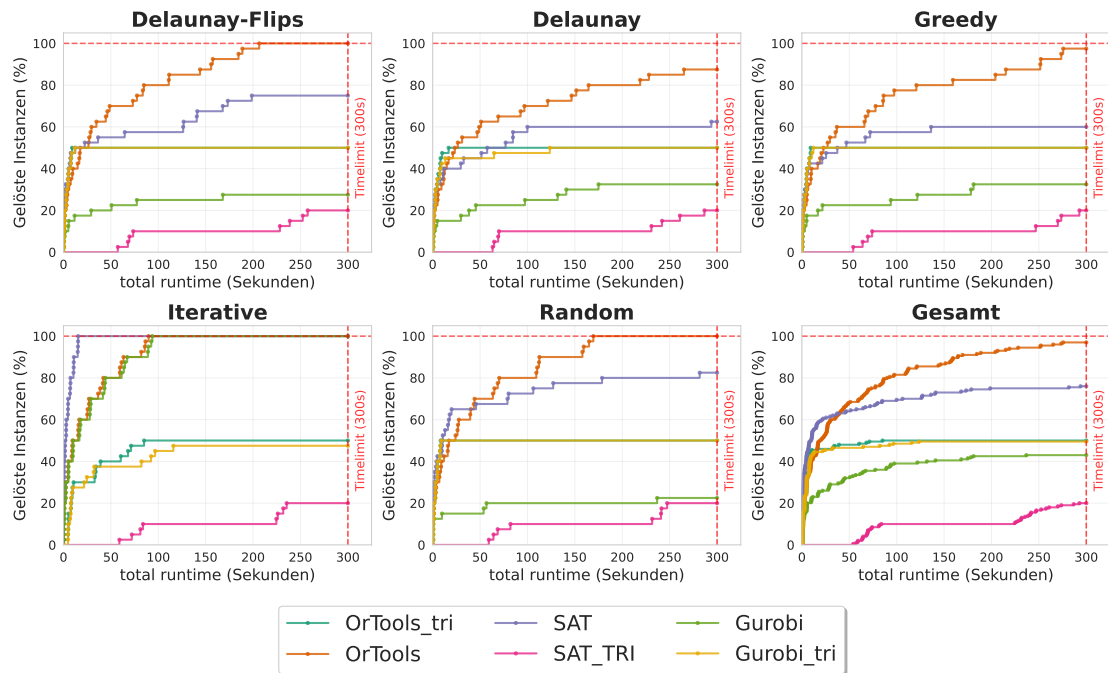


Abbildung 6.3.: In dieser Abbildung werden für die sechs verschiedenen Ansätze die besten Solver verglichen. Für PySAT und Gurobi ist die beste Methode das Fixieren der Kanten bei der Kantenformulierung. Bei OrTools hingegen ist die beste Methode die alternative Kantenformulierung bei der Kantenformulierung. Insgesamt erzielt OrTools die besten Ergebnisse, gefolgt von PySAT. Gurobi ist in allen Fällen mit Abstand schlechter als PySAT und OrTools. PySAT ist in leichten Instanzen wie Iterativ und Random allerdings besser als OrTools. Es ist auch klar zu erkennen, dass die Dreiecksformulierung bei allen Solvern schlechtere Ergebnisse erzielt als die Kantenformulierung.

dass beim Fixieren so viele Kanten durch den Solver indirekt durch andere Bedingungen ausgeschlossen werden können, dass ab einer bestimmten Menge an fixierten Kanten die resultierenden Teilinstanz gleich groß sind.

6.8. Größerer Timeout

In diesem Experiment wird geprüft, welche Instanzen der Solver lösen kann, wenn das Zeitlimit von 5 Minuten auf 30 Minuten erhöht wird. Dafür wurden Instanzen mit zunehmender Größe generiert, bis der Solver diese nicht mehr lösen konnte. Die Knotenzahl lag am Ende zwischen 130 und **200** und es wurden auf dem Ryzen 9 getestet. Es wird erneut der beste Solver aus Abschnitt 6.6 verwendet. In Abbildung F.1 ist zu erkennen, dass der Solver Instanzen mit bis zu **170** Knoten lösen konnte.

6.9. Ja/Nein getrennt

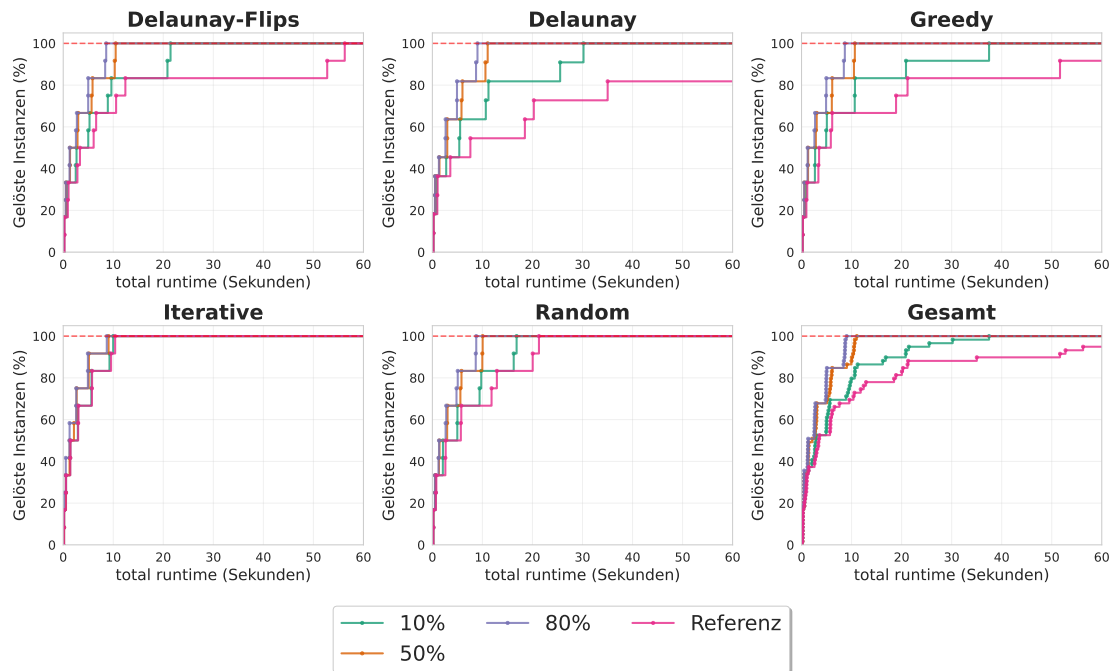


Abbildung 6.4.: In diesem Experiment wird die theoretische Ausschließung von Kanten untersucht. Für das Experiment wurde OrTools mit Grad und Überschneidungsbeschränkung als Referenz und zusätzlich OrTools mit 10, 50 und 80 Prozent der benötigten Kanten ausgeschlossen verwendet. In dem Diagramm wurde der betrachtete Zeitraum auf 60 Sekunden begrenzt da hier alle relevanten Informationen zu sehen sind. Es ist klar zu erkennen, dass der Referenz-Solver die schlechtesten Ergebnisse erzielt. Auch die anderen Solver sind in der erwarteten Reihenfolge. Nur 50 Prozent und 80 Prozent sind deutlich besser als die andern Ansätze.

6.9. Ja/Nein getrennt

In diesem Experiment wird untersucht, ob der SAT Solver bei Ja-Instanzen oder bei Nein-Instanzen schneller ist. Dafür werden die Daten aus Abschnitt 6.8 wiederverwendet. In Abbildung G.1 ist zu erkennen, dass der Solver bei Nein-Instanzen schneller ist und auch größere Instanzen lösen kann. Insgesamt konnte der Nein-Instanzen bis zu **170** Knoten lösen, während Ja-Instanzen nur bis zu **160** Knoten gelöst werden konnten.

6.10. Zielfunktion

In diesem Experiment werden die beiden Zielfunktion aus Abschnitt 3.2 verglichen. Es sei noch zu erwähnen, dass der Flips-Ansatz vorab getestet wurde. Dieser Ansatz hat im Maximalfall 8 Knoten geschafft. Da die andern beiden Zielfunktionen auf Instanzen mit

6. Auswertung

120 Knoten getestet werden, ist der Flips Ansatz nicht mehr relevant. Das Experiment wurde auf dem Ryzen 9 durchgeführt.

Im folgenden werden Ja-Instanzen und Nein-Instanzen getrennt betrachtet. Bei Ja-Instanzen ist interessant, wie schnell die Instanz gelöst werden kann und ab wann gute Ergebnisse erzielt werden. Bei den Nein-Instanzen ist hingegen interessanter wie nah der Solver an eine Lösung kommt. Bei beiden Instanzen wird ein Referenz-Solver verwendet. Dieser betrachtet das Problem weiterhin als Entscheidungsproblem. Ausgewählt wurde der OrTools Solver mit den in Abschnitt 6.6 gefundenen Bedingungen. Die Zielfunktionen nutzen ebenfalls diese Bedingungen, nur die Gradbeschränkung wird durch die Zielfunktion ersetzt.

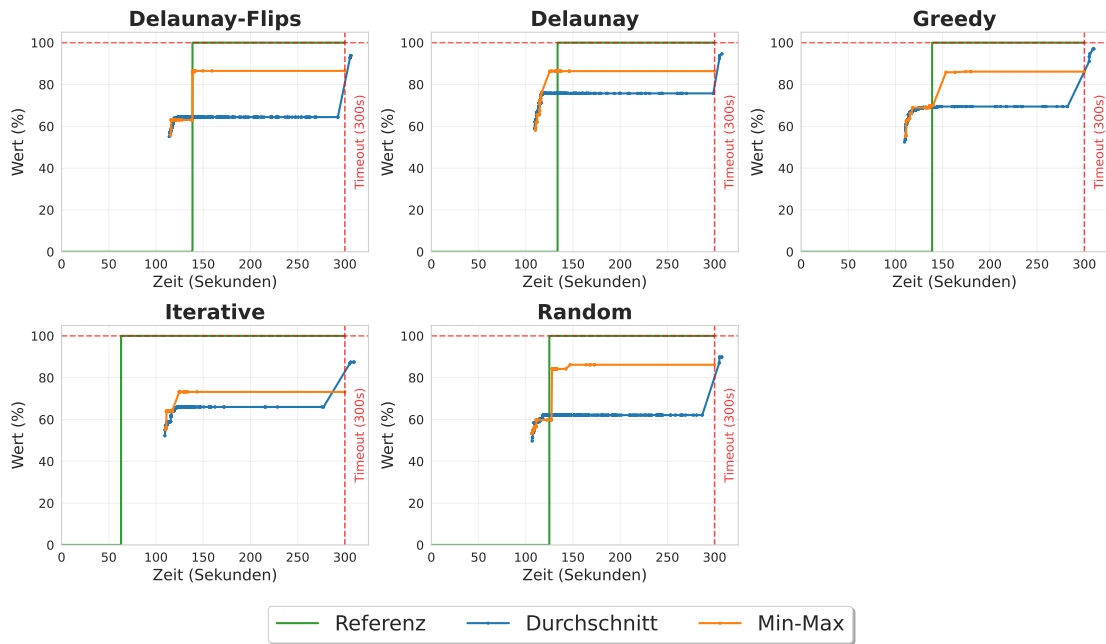


Abbildung 6.5.: Eine Nein-Instanz mit 120 Knoten. Hier werden zwei verschiedene Zielfunktionen mit einem Referenzsolver aus Abschnitt 6.6 verglichen. Es ist zu erkennen, dass der Durchschnittsansatz in den meisten Fällen bessere Ergebnisse liefert als der Min-Max-Ansatz. Oft findet er erst beim Timeout eine Verbesserung, diese ist jedoch besser als die des Min-Max-Ansatzes. In den meisten Fällen stagniert der Min-Max-Ansatz bei etwas über 80 Prozent. Der Min-Max-Ansatz bricht vermutlich früher ab, da er aus seiner Sicht ein Optimum gefunden hat. Der Referenzsolver erkennt nur bei Greedy und Iterative früher, dass es sich um eine Nein-Instanz handelt. In allen anderen Fällen benötigt er mindestens genauso lange wie der Min-Max-Ansatz.

In Abbildung H.1 zeigt sich, dass bei Ja-Instanzen der Min-Max-Ansatz besser abschneidet. Bei Nein-Instanzen erzielt der Durchschnittsansatz die besseren Ergebnisse, dies ist

6. Auswertung

Der erste Schritt war es, die Teilinstanzen, mit denen die *Propagation* aufgerufen wird, zu visualisieren. In Abbildung 6.6 ist eine beispielhafte Teilinstanz dargestellt. Bei der Visualisierung wurde festgestellt, dass es häufig zwei Arten von Teilinstanzen gibt. Die erste Variante bildet ein Konstrukt, das eine Verbindung zum Rand besitzt. In diesem Konstrukt ist meist eine gültige DCT vorhanden. Die zweite Variante besteht aus einzelnen Kanten, die isoliert in einer Fläche liegen.

6.11.2. Relative Anzahl Erfolgreicher Propagation

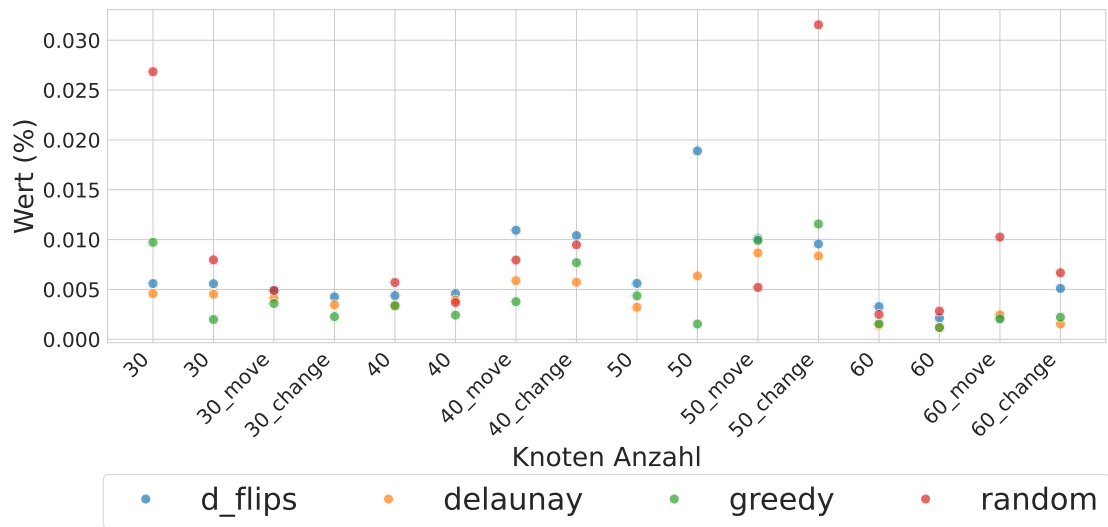


Abbildung 6.7.: In der Abbildung sind die Relativen Anzahlen der Erfolgreichen *Propagation* zu sehen. Dafür wird auf der Y-Achse die Anzahl der Erfolgreichen *Propagation* geteilt durch die Anzahl der *Propagation* Aufrufe dargestellt. Auf der X-Achse sind die Knotengrößen aufsteigend dargestellt. Dabei gibt es immer zwei Ja-Instanzen und zwei Nein-Instanzen pro Knotengröße. Bei den Nein-Instanzen wurde jeweils eine *move* und eine *change* Instanz genutzt. Es fällt auf das vor allem Random Instanzen sehr viele Erfolgreiche *Propagation* haben. Greedy und Delaunay hingegen haben nur wenige Erfolgreiche *Propagation*. Iterativ hatte anscheinend keine Erfolgreichen *Propagation* da die Instanzen zu einfach sind. Nicht ganz deutlich aber zu erkennen ist das Nein-Instanzen mehr Erfolgreiche *Propagation* haben als Ja-Instanzen.

FA: Größere Insanzen laufen lassen, wahrscheinlich nicht viel größere da es keinen Timeout gibt und das dass Testen schwierig macht

In diesem Experiment wird betrachtet wie viele Erfolgreiche *Propagation* also jene, welche eine Kante ausschließen können, in einer Instanz möglich sind. Dafür werden alle Erfolgreichen Aufrufe gezählt und diese Zahl durch die Anzahl der *Propagation* Aufrufe

geteilt. Dabei wird in diesem Experiment nicht die Laufzeit gemessen da durch die vielen Kanten welche getestet werden die Laufzeit stark beeinflusst wird. Eine *Propagation* soll im Normalfall sehr schnell entscheiden welche Literale mit der aktuellen Belegung noch gesetzt werden können. In diesem Experiment geht es nur darum zu bestimmen ob es sinnvoll ist eine *Propagation* mit diesem Ansatz zu nutzen. Bei diesem Experiment wurden Instanzen der Größe 30 bis 60 Knoten betrachtet. Das Experiment wurde auf dem Ryzen 9 durchgeführt.

In Abbildung 6.7 sind diese relativen Anzahlen pro Instanz zu sehen. Dabei fällt auf dass vor allem Random Instanzen sehr viele erfolgreiche *Propagation* haben. Es lässt sich vermuten dass Nein-Instanzen mehr erfolgreiche *Propagation* haben als Ja-Instanzen. Dabei kann es sich aber auch um einen Zufall handeln da nur wenige verschiedene Instanzen betrachtet wurden.

6.12. Schwierigkeit der Instanzen

In diesem Abschnitt wird probiert die Schwierigkeit der Instanzen zu bestimmen. Aus den vorigen Experimenten ist bekannt dass *Delaunay* und *Greedy* die schwereren Instanzen sind, *Delaunay Flips* im Mittelfeld und *Iterativ* nach *Random* die einfachsten Instanzen sind. Im ersten Teil wird geschaut ob die Anzahl der Lösungen pro Instanz einen Einfluss auf die Schwierigkeit hat. Danach im zweiten Teil wird geschaut wie der Grad verteilt ist. Also wie oft jeder Grad vertreten wird. Im letzten Teil wird geschaut ob ein Zusammenhang zu den Kantenlängen besteht. Für alle drei Teile werden Instanzen der Größe 30 bis 120 Knoten betrachtet.

6.12.1. Anzahl Lösungen

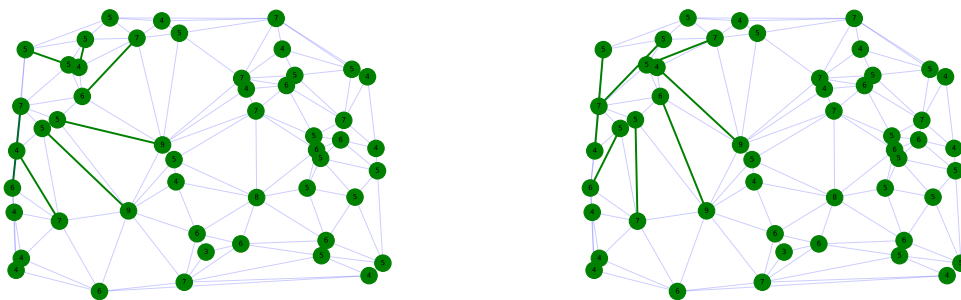


Abbildung 6.8.: In dem Bild sind zwei verschiedene Lösungen für eine Instanz zu sehen. In Grün sind die Kanten zu sehen welche in der Instanz unterschiedlich sind. In Blau die gleichen Kanten. Die Zahlen in den Knoten geben den Sollgrad an.

6. Auswertung

In table E.1 ist zu sehen das fast alle Instanzen nur eine Lösungen haben. Vorallem ist aber zu sehen das es keinen Zusammenhang zwischen der Anzahl der Lösungen und der Instanze Art gibt. Es kann also davon ausgegangen werden das die Anzahl der Lösungen keinen Einfluss auf die Schwierigkeit der Instanzen hat. Interessanter weiße ist zu sehen das die Kanten welche in einer Instanze unterschiedliche sind klar einem Bereich zugeordnet werden könne. Wie man in Abbildung 6.8 sieht, sind die Kanten in der linken Hälfte der Instanz unterschiedlich, während die rechte Hälfte keine Unterschiede aufweist.

6.12.2. Gradverteilung

In diesem Experiment wird Probiert anhand der Gradverteilung der Knoten zu erkennen ob es einen Zusammenhang zwischen der Schwierigkeit und dem Grad gibt. Dafür werden die Ja-Instanzen der Größe 30 bis 120 Knoten betrachtet. In Abbildung 6.9 ist die

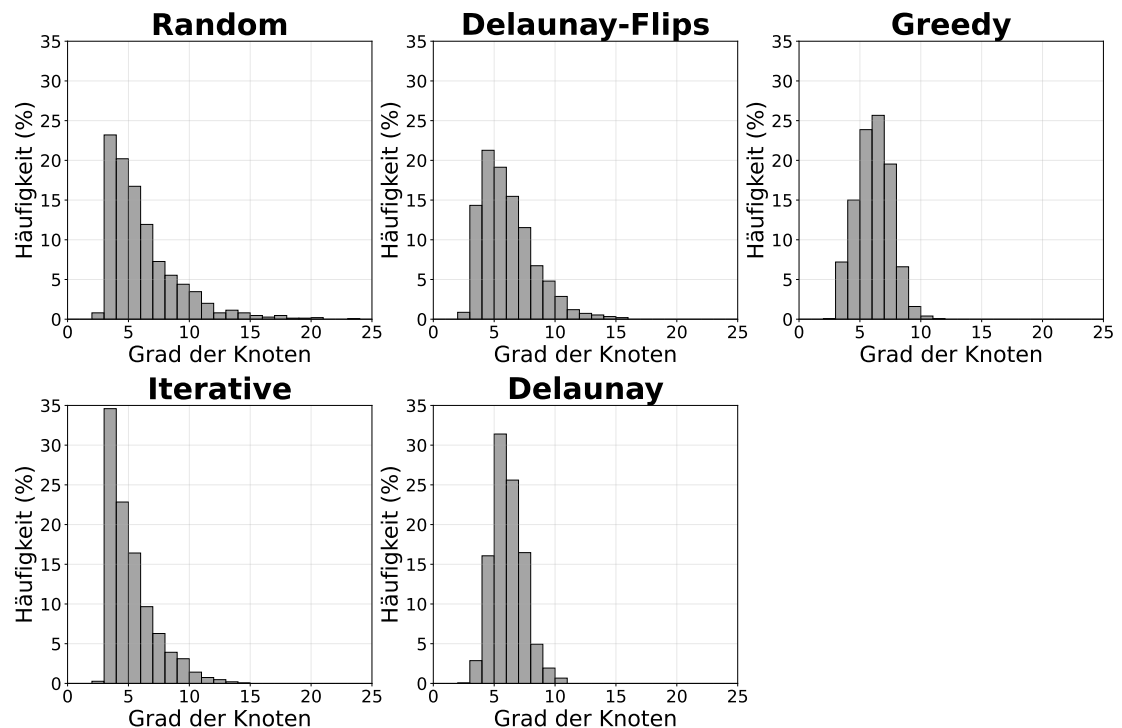


Abbildung 6.9.: In dieser Abbildung ist die Gradverteilung der Knoten pro Instanz zu sehen. Dabei wird auf der X-Achse der Grad und auf der Y-Achse die Häufigkeit der Knoten mit diesem Grad dargestellt.

Gradverteilung der Knoten zu sehen. Es lässt sich vermuten das es einen Zusammenhang zwischen der Schwierigkeit und dem Grad gibt. Es ist zu erkennen das Vorallem Random und Iterativ eine Rechtsschiefe Verteilung haben. Dazu passend haben Delaunay und Greedy eher eine zentrierte Verteilung. Auch das Delaunay-Flips Rechtsschiefer als Delaunay und Greedy ist würde passen, da Delaunay-Flips einfacher als Delaunay ist. Es

lässt sich also Anhand dieser Beispiel vermuten das einfachere Instanzen eher kleinere Grade haben und schwierigere Instanzen mehr Verteile mittlere Grade.

6.12.3. Verteilung der Kantenlängen

In diesem Experiment wird ebenfalls probiert einen Zusammenhang zwischen der Schwierigkeit und einer Eigenschaft zu finden. In diesem Fall werden die Kantenlängen betrachtet. Allerdings müssen diese Normiert werden da sonst durch weit auseinander liegende Knoten die Verteilung verzerren würden. Die Normierung erfolgt durch das Teilen durch die maximale Kantenlänge. Das heißt für jede Instanz wird die maximale Kantenlänge bestimmt und alle Kantenlängen durch diese geteilt. Für dieses Experiment wurden nur Ja-Instanzen der Größe 30 bis 80 Knoten betrachtet.

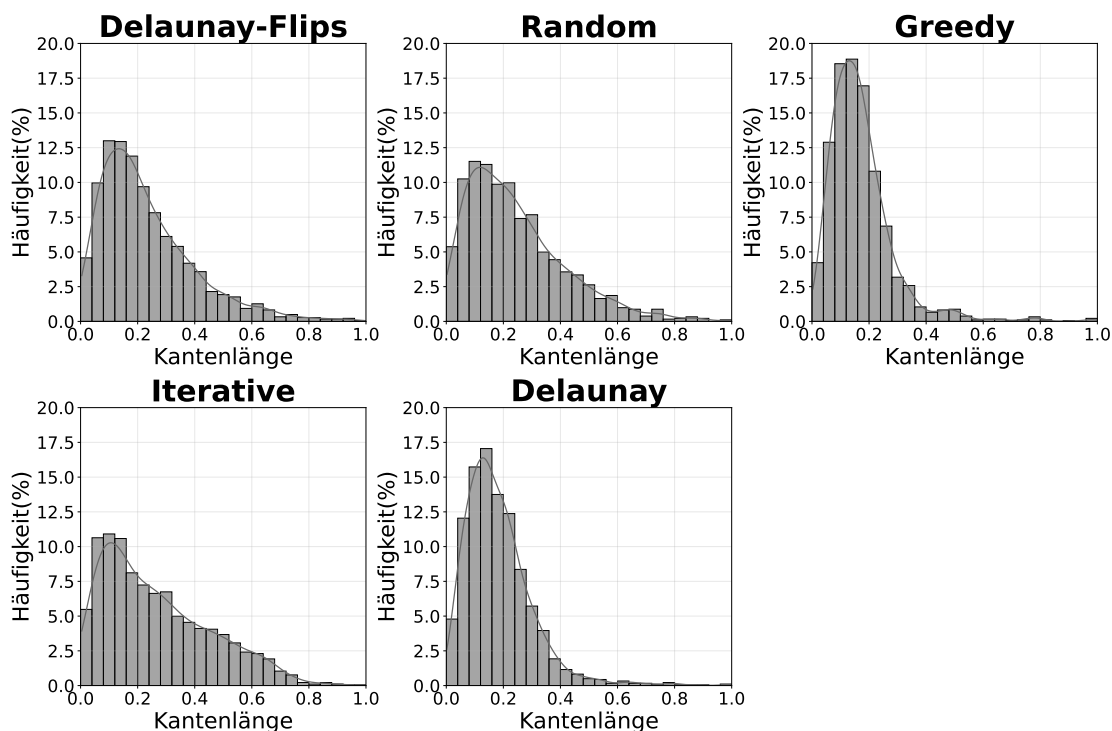


Abbildung 6.10.: In dieser Abbildung ist die Verteilung der Kantenlängen pro Instanz zu sehen. Dabei sind die Kantenlängen normiert, sodass sie zwischen 0 und 1 liegen. Dafür wurde alle Kantenlängen durch die maximale Kantenlänge geteilt. Auf der X-Achse ist die normierte Kantenlänge und auf der Y-Achse die Häufigkeit der Kanten mit dieser Länge zu sehen.

Auch hier lässt sich eine Verbindung zwischen der Schwierigkeit und der Verteilung erkennen. Leichtere Instanzen haben eher lange Kanten als schwere Instanzen. So sind bei schweren Instanzen wie Greedy und Delaunay die Kantenlängen eher links also bei

6. Auswertung

den kurzen Kanten angesiedelt. Leichte Instanzen wie Iterativ hingegen haben deutlich weniger kurze Kante und dafür mehr längere Kanten.

6.12.4. Zusammenfassung

In diesem Abschnitt wurde festgestellt, dass die Anzahl der Lösungen keinen Einfluss auf die Schwierigkeit hat. Die Gradverteilung und die Kantenlängenverteilung hingegen scheinen einen Einfluss auf die Schwierigkeit zu haben. Hierbei deuten viele kurze Kanten und verteilte Grade auf eine schwere Instanz hin. Hingegen verteilte längere Kanten und wenige hohe Grade auf eine leichte Instanz.

7. Conclusion (and Future Work)

In dieser Arbeit wurde das neuartige Problem der gradbeschränkten Triangulierungen (Degree Constrained Triangulations, DCT) untersucht. Dabei wurde es als Entscheidungsproblem sowie als Optimierungsproblem betrachtet. Für das Entscheidungsproblem wurde eine Kanten basierte und eine dreiecksbasierte Kodierung entwickelt. Hierbei wurde ein Fokus auf das fixieren und ausschließen von Kanten gelegt. Beide wurden als Mathematische und KNF Formulierungen umgesetzt. Diese Formulierungen wurden mit PySAT, Gurobi und OR-Tools implementiert. Diese Implementierungen wurden an zuvor generierten Instanzen getestet. Es wurden fünf verschiedene Instanztypen entwickelt, um sowohl lösbare als auch unlösbare Fälle zu erzeugen. Die experimentelle Evaluation zeigte, dass insbesondere OR-Tools die besten Ergebnisse liefern konnte und Instanzen bis zu einer Größe von 170 Knoten löst. Zusätzlich wurde mit dem CDCL Ansatz eine Theoretische Möglichkeit gefunden den Lösungsprozess zu optimieren. Dennoch bleibt die Skalierbarkeit auf große Instanzen eine Herausforderung, da die vorgestellten Verfahren vor allem für kleinere Punktmengen praktikabel sind. Darüber hinaus wurde festgestellt, dass die Struktur der Instanzen, insbesondere die Verteilung der Kantenlängen und Knotengrade, einen erheblichen Einfluss auf die Schwierigkeit der Instanz hat.

Ein vielversprechender Ausblick besteht darin, die vorgestellten Verfahren hinsichtlich ihrer Skalierbarkeit zu verbessern, etwa durch leistungsfähigere Heuristiken oder hybride Ansätze, die exakte und heuristische Methoden kombinieren. Zudem wäre es interessant bei der Instanzengenerierung auf normalverteilte Triangulierungen zu achten, um einheitliche Szenarien abzubilden.

FA: Dreiecke mehr nennen

FA: @Michael oder Phillip: Habt ihr noch Ideen was in den Schluss muss?

Literaturverzeichnis

- [1] Roberto Asín, Robert Nieuwenhuis, Albert Oliveras, and Enric Rodríguez-Carbonell. Cardinality networks and their applications. In *International Conference on Theory and Applications of Satisfiability Testing*, pages 167–180. Springer, 2009.
- [2] Olivier Bailleux and Yacine Boufkhad. Efficient cnf encoding of boolean cardinality constraints. In *International conference on principles and practice of constraint programming*, pages 108–122. Springer, 2003.
- [3] Kenneth E Batchner. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, spring joint computer conference*, pages 307–314, 1968.
- [4] Armin Biere. Arminbiere/cadical: Cadical sat solver, 2019. URL: <https://github.com/arminbiere/cadical>.
- [5] Prosenjit Bose and Ferran Hurtado. Flips in planar graphs. *Computational Geometry*, 42(1):60–80, 2009. URL: <https://www.sciencedirect.com/science/article/pii/S0925772108000370>, doi:10.1016/j.comgeo.2008.04.001.
- [6] Basudeb Datta and Subhojoy Gupta. Degree-regular triangulations of surfaces. *arXiv preprint arXiv:1711.01247*, 2017.
- [7] Ana Maria de Almeida, Pedro Martins, and Maurício C de Souza. Min-degree constrained minimum spanning tree problem: complexity, properties, and formulations. *International Transactions in Operational Research*, 19(3):323–352, 2012.
- [8] Mark de Berg, Marc van Kreveld, Mark Overmars, and Otfried Cheong Schwarzkopf. *Delaunay Triangulations*, pages 183–210. Springer Berlin Heidelberg, Berlin, Heidelberg, 2000. doi:10.1007/978-3-662-04245-8_9.
- [9] Sándor P Fekete, Phillip Keldenich, and Michael Perk. Exact algorithms for minimum dilation triangulation. *arXiv preprint arXiv:2502.18189*, 2025.
- [10] Sándor P Fekete, Samir Khuller, Monika Klemmstein, Balaji Raghavachari, and Neal Young. A network-flow technique for finding low-weight bounded-degree spanning trees. *Journal of Algorithms*, 24(2):310–324, 1997.
- [11] Laxmi P Gewali and Bhaikaji Gurung. Degree aware triangulation of annular regions. In *Information Technology-New Generations: 15th International Conference on Information Technology*, pages 683–690. Springer, 2018.

- [12] F. Hurtado, M. Noy, and J. Urrutia. Flipping edges in triangulations. In *Proceedings of the Twelfth Annual Symposium on Computational Geometry*, SCG '96, page 214–223, New York, NY, USA, 1996. Association for Computing Machinery. doi:10.1145/237218.237367.
- [13] J. Knowles and D. Corne. A new evolutionary approach to the degree-constrained minimum spanning tree problem. *IEEE Transactions on Evolutionary Computation*, 4(2):125–134, 2000. doi:10.1109/4235.850653.
- [14] Donald E. Knuth. *The Art of Computer Programming, Volume 4, Fascicle 6: Satisfiability*. Addison-Wesley Professional, 1st edition, 2015.
- [15] Mohan Krishnamoorthy, Andreas T. Ernst, and Yazid M. Sharaiha. Comparison of algorithms for the degree constrained minimum spanning tree. *Journal of Heuristics*, 7(6):587–611, 2001. doi:10.1023/A:1011977126230.
- [16] Antonio Morgado, Alexey Ignatiev, and Joao Marques-Silva. Mscg: Robust core-guided maxsat solving: System description. *Journal on Satisfiability, Boolean Modelling and Computation*, 9(1):129–134, 2014.
- [17] Oleg R Musin. Properties of the delaunay triangulation. In *Proceedings of the thirteenth annual symposium on Computational geometry*, pages 424–426, 1997.
- [18] Toru Ogawa, Yangyang Liu, Ryuzo Hasegawa, Miyuki Koshimura, and Hiroshi Fujita. Modulo based cnf encoding of cardinality constraints and its application to maxsat solvers. In *2013 IEEE 25th International Conference on Tools with Artificial Intelligence*, pages 9–17. IEEE, 2013.
- [19] Micha Sharir, Adam Sheffer, and Emo Welzl. On degrees in random triangulations of point sets. In *Proceedings of the twenty-sixth annual symposium on Computational geometry*, pages 297–306, 2010.
- [20] Carsten Sinz. Towards an optimal cnf encoding of boolean cardinality constraints. In *International conference on principles and practice of constraint programming*, pages 827–831. Springer, 2005.

A. Permutation

FA: alle drei Bilder haben den gleichen Anfang ich weiß aber nicht wie ich das anders machen kann

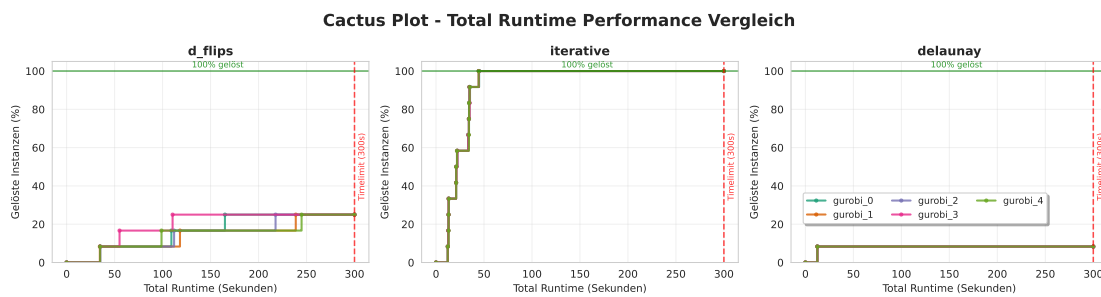


Abbildung A.1.: In diesem Diagramm wird untersucht ob Gurobi bei verschiedenen Permutation unterschiedliche Laufzeiten hat. Dazu wird jede Permutation als eine Linie dargestellt. Es ist zu erkennen das vorallem bei Delaunay Flips Linen verschiedene Laufzeiten haben, dementsprechend hat Gurobi bei verschiedenen Permutation unterschiedliche Laufzeiten.

A. Permutation

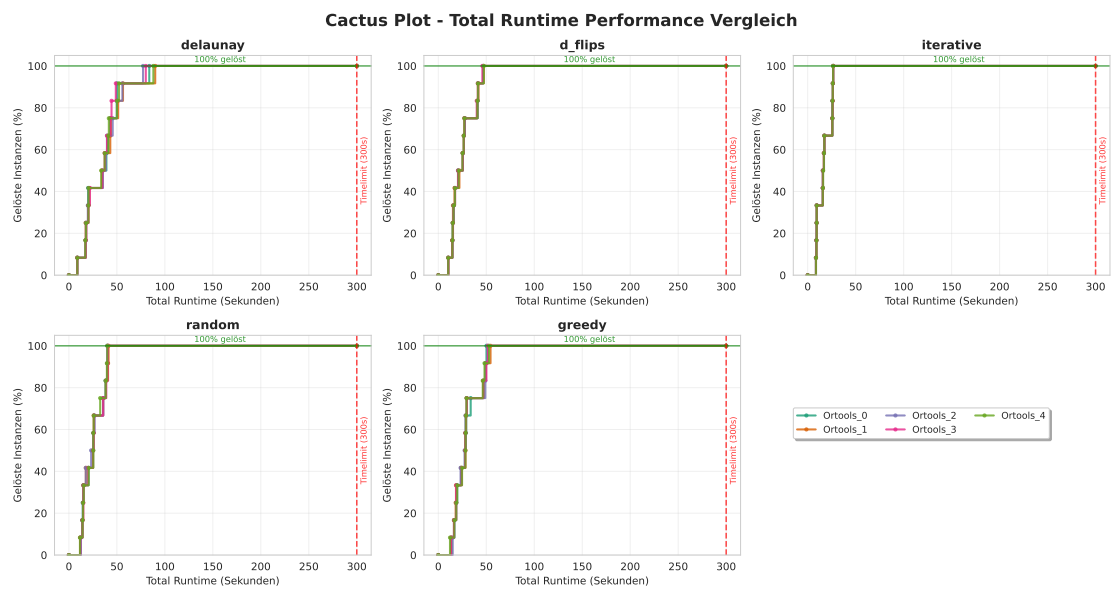


Abbildung A.2.: In diesem Diagramm wird untersucht ob OrTools bei verschiedenen Permutation unterschiedliche Laufzeiten hat. Dazu wird jede Permutation als eine Linie dargestellt. Es ist zu erkennen das die meisten Instanzen die gleichen Laufzeiten haben. Nur bei Delaunay sind die Laufzeiten leicht unterschiedlich.

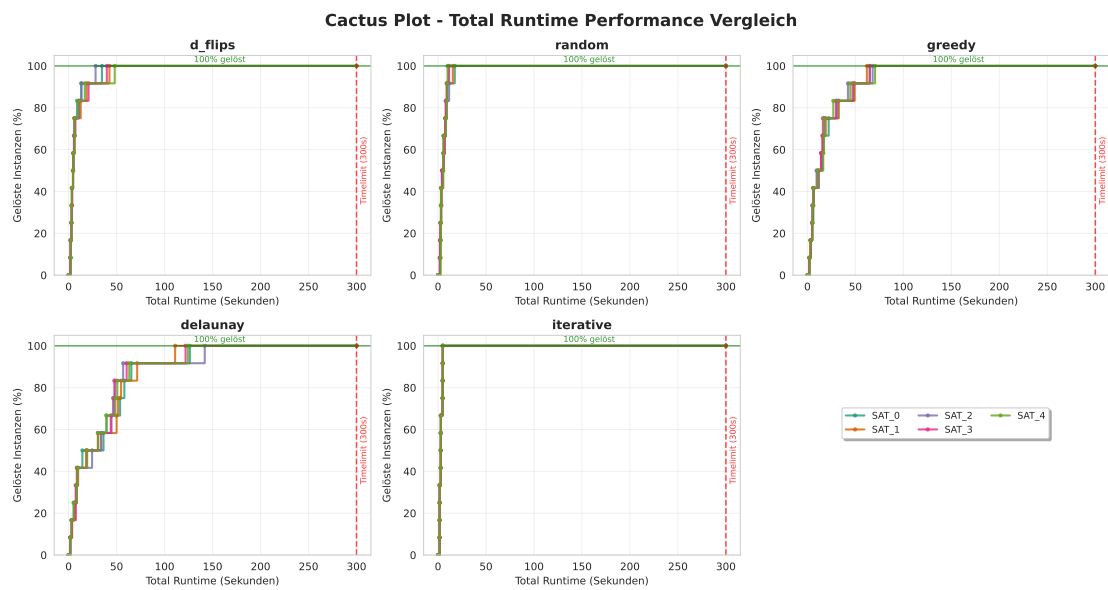


Abbildung A.3.: In diesem Diagramm wird untersucht ob PySAT bei verschiedenen Permutation unterschiedliche Laufzeiten hat. Dazu wird jede Permutation als eine Linie dargestellt. Bei Random und Iterativ unterscheiden sich die Laufzeiten kaum. Bei den anderen drei Instanzen vor allem bei Delaunay unterscheiden sie sich aber.

B. Mehrmalige Durchläufe

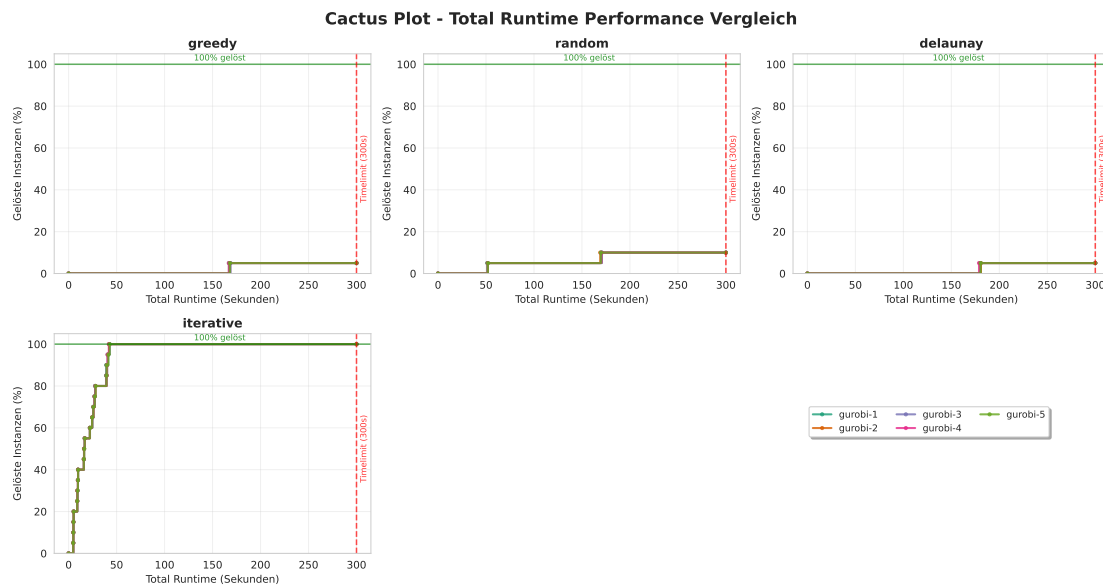


Abbildung B.1.: In diesem Diagramm wird untersucht ob Gurobi bei den gleichen Argumenten und den gleichen unveränderten Instanzen unterschiedliche Laufzeiten hat. Dazu wird jede Durchgang als eine Linie dargestellt. Es ist zu sehen das alle Linien fast identisch sind. Daraus lässt sich folgen das Gurobi bei gleichen Bedingungen die gleichen Ergebnisse liefert.

B. Mehrmalige Durchläufe

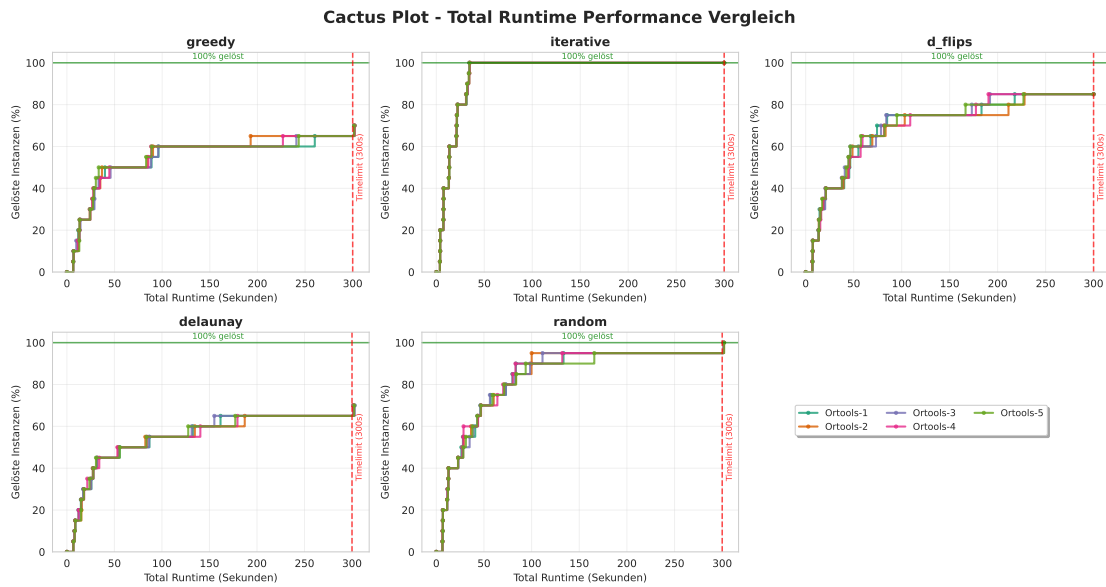


Abbildung B.2.: In diesem Diagramm wird untersucht ob OrTools bei den gleichen Argumenten und den gleichen unveränderten Instanzen unterschiedliche Laufzeiten hat. Dazu wird jede Durchgang als eine Linie dargestellt. Es fällt auf dass außer bei Iterativ alle Linien unterschiedlich sind. Vorallem bei Delaunay_Flips und Greedy sind die Unterschiede groß. Daraus lässt sich folgen dass OrTools bei gleichen Bedingungen unterschiedliche Ergebnisse liefert.

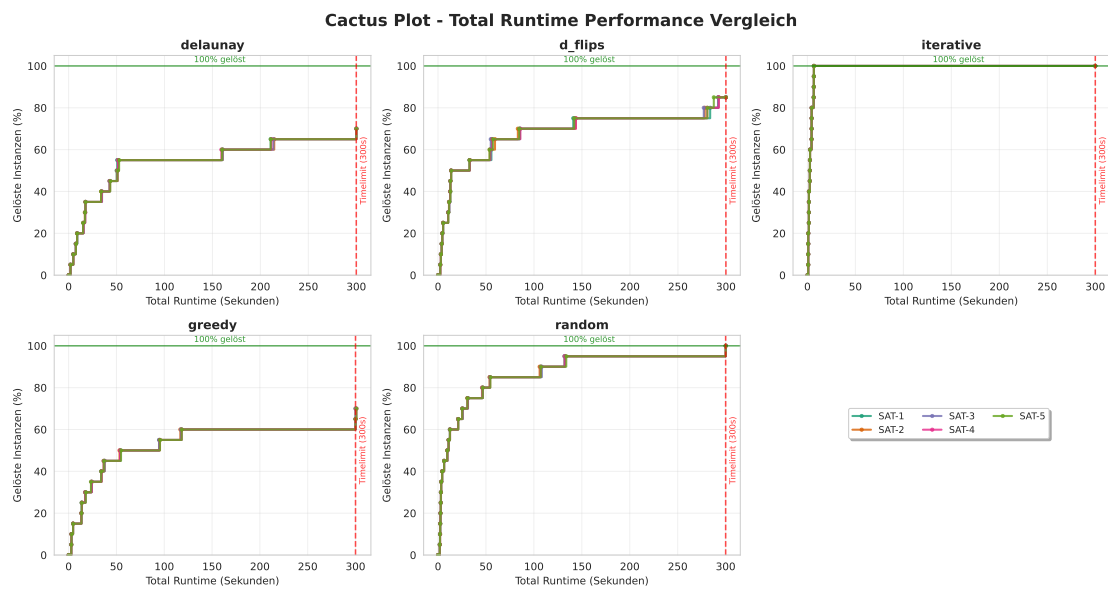
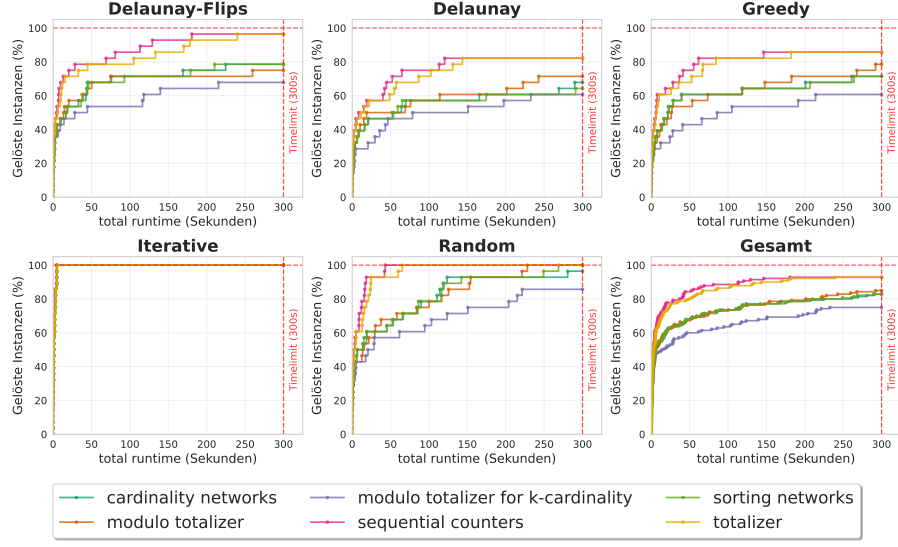


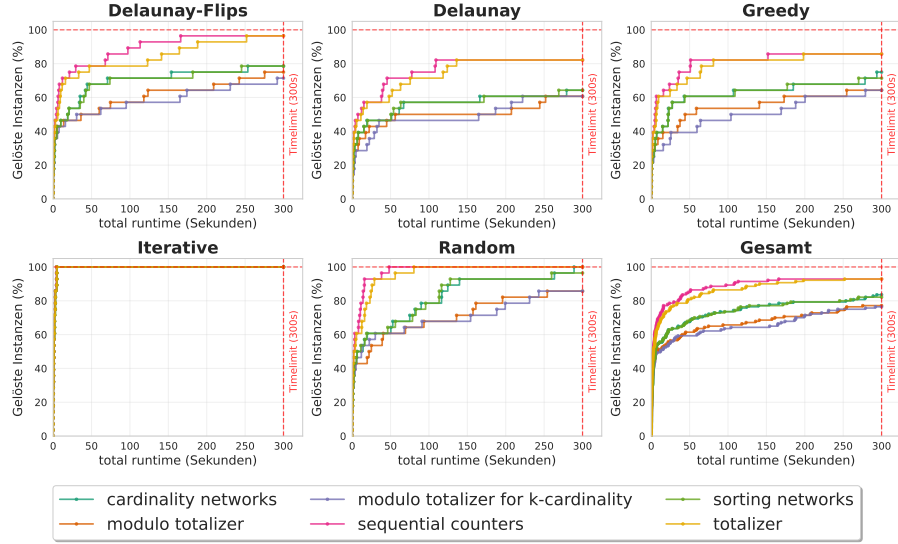
Abbildung B.3.: In diesem Diagramm wird untersucht ob PySAT bei den gleichen Argumenten und den gleichen unveränderten Instanzen unterschiedliche Laufzeiten hat. Dazu wird jede Durchgang als eine Linie dargestellt. Es ist zu sehen das alle Linien fast identisch sind. Daraus lässt sich folgen das PySAT bei gleichen Bedingungen die gleichen Ergebnisse liefert.

C. SAT Degree Auswertung

C. SAT Degree Auswertung



(a) Exakte Sollgrad: $\sum x_{e_i} = \deg(i) \forall i \in V$.



(b) Mindest Sollgrad: $\sum x_{e_i} \geq \deg(i) \forall i \in V$

Abbildung C.1.: In diesem Diagrammen werden verschiedene Codierungen für Kardinalitäten in einer KNF verglichen. Diese Kardinalität werden dazu genutzt das in einer KNF für jeden Knoten alle inzidenten Kanten gleich bzw. mindestens dem Sollgrad entsprechen müssen. Dabei wird zum einem der exakte Sollgrad (a) und zum anderen der minimale Sollgrad (b) betrachtet. In beiden Diagrammen ist klar zu erkennen das die *sequential counters* Methode am besten funktioniert, gefolgt von *totalizer*.

D. Alle Parameter

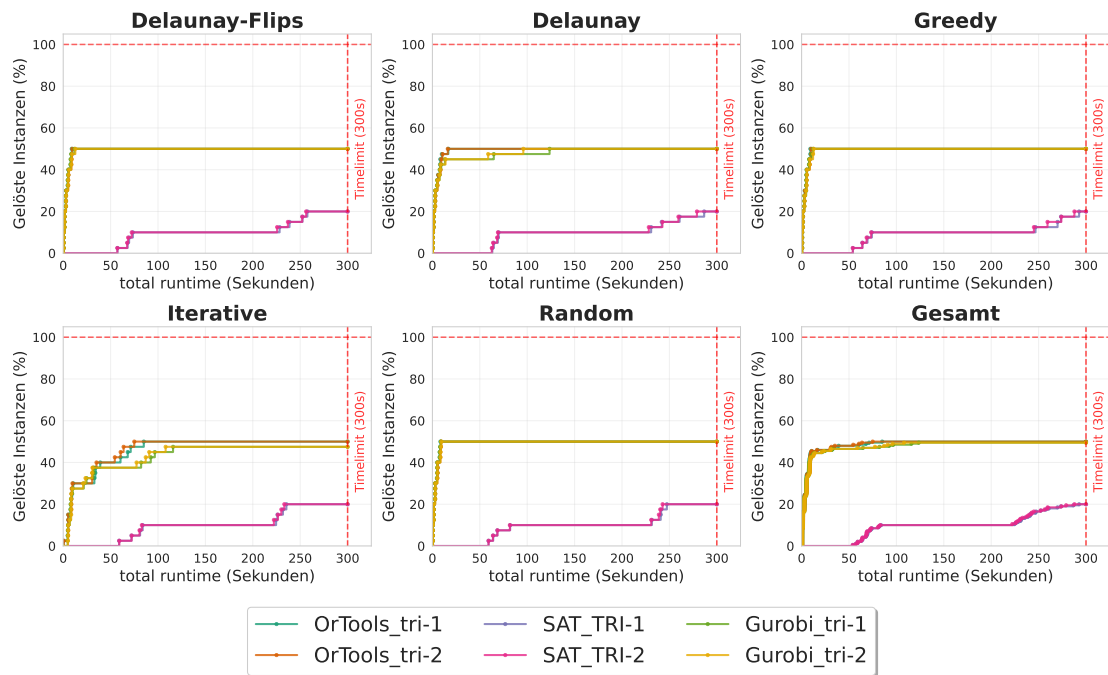


Abbildung D.1.: Eval1 Dreiecke

D. Alle Parameter

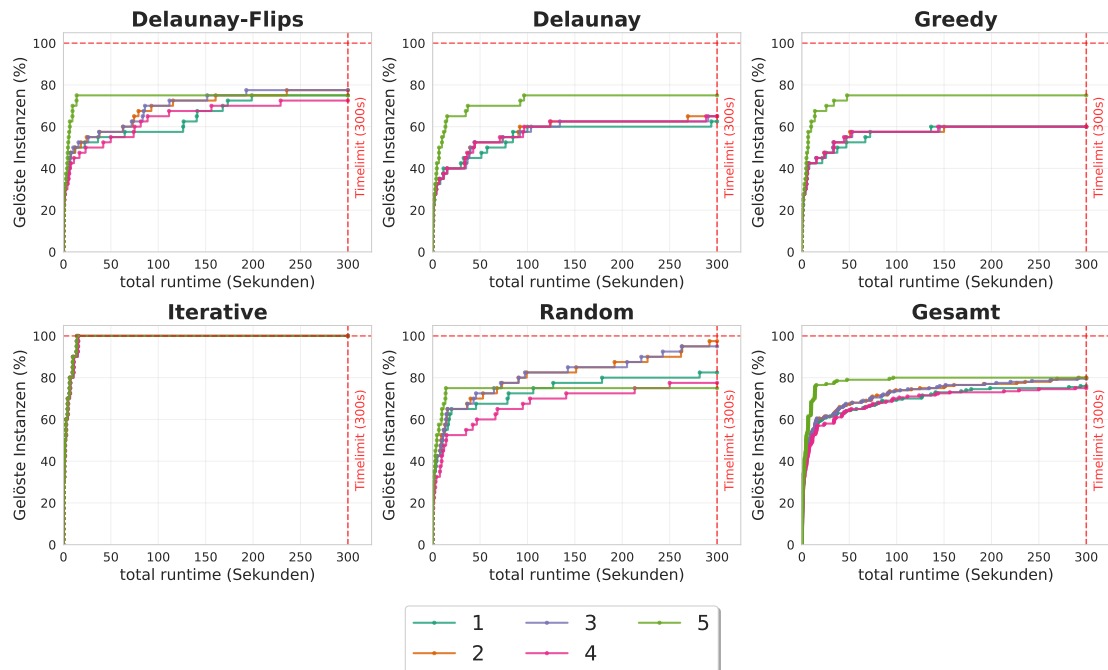


Abbildung D.2.: In diesem Experiment werden fünf verschiedene Optimierungsansätze für PySAT getestet. Hierbei wird untersucht welche Optimierungen besser sind als der Normale Ansatz. In diesem Diagramm ist klar erkennbar das der einzige Ansatz welcher besser ist das fixieren der Hülle ist. Allerdings ist der Unterschied nicht sehr groß. Alle andern Optimierungen sind deutlich schlechter.

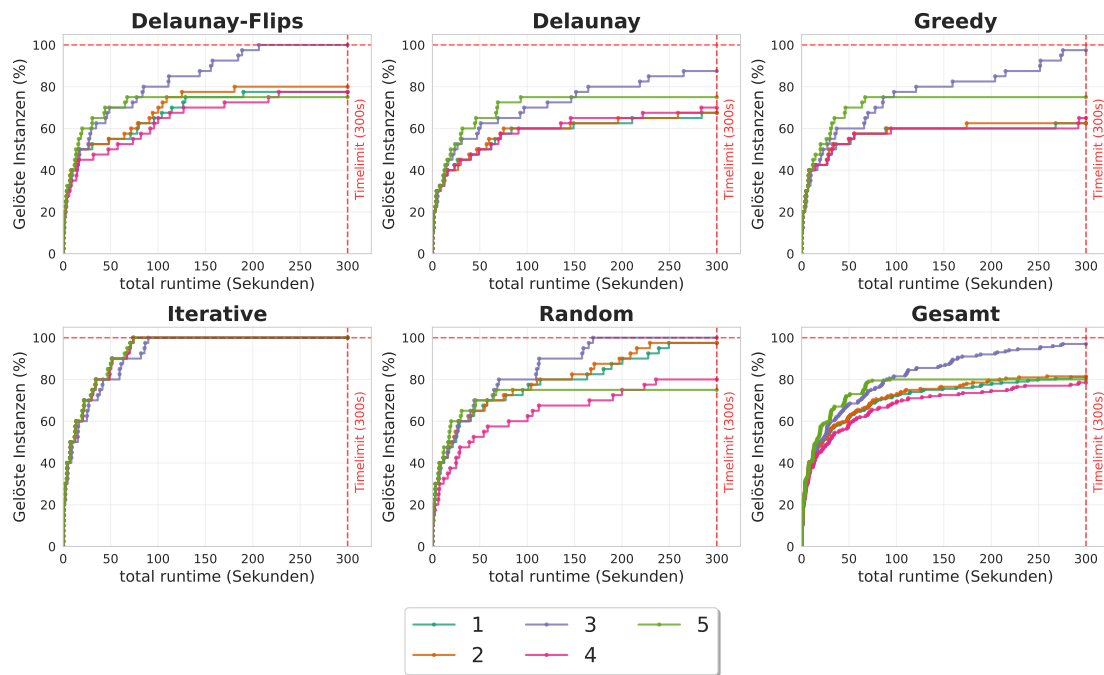


Abbildung D.3.: In diesem Experiment werden fünf verschiedene Optimierungsansätze für OrTools getestet. Hierbei wird untersucht welche Optimierungen besser sind als der Normale Ansatz. Es zeigt sich das das fixieren der Hülle ähnliche teilweise bessere Ergebnisse als der normale Ansatz liefert. Das fixieren und ausschließen der Kanten ist allerdings deutlich schlechter. Die alternative Kanten bedingung gibt den größten Unterschied. Hier ist in der gesamt übersicht zu erkennen das dieser Ansatz deutlich besser ist als alle anderen. Nur in den Iterativen Instanzen ist der normale Solver am besten.

D. Alle Parameter

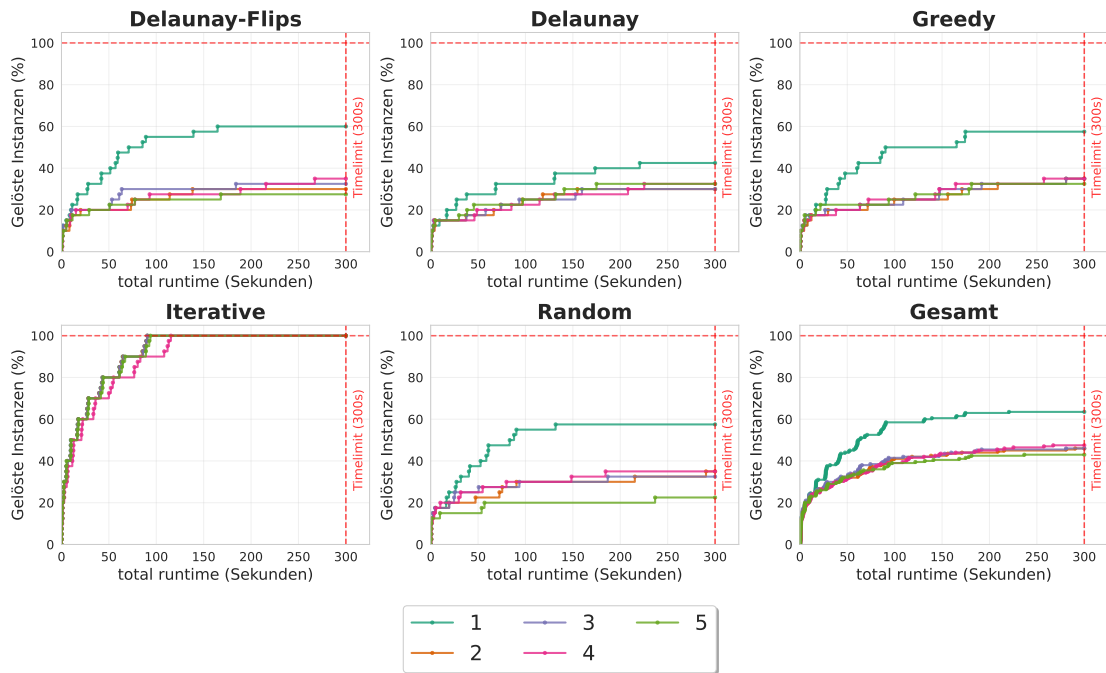


Abbildung D.4.: In diesem Experiment werden fünf verschiedene Optimierungsansätze für Gurobi getestet. Hierbei wird untersucht welche Optimierungen besser sind als der Normale Ansatz. Es ist zu erkennen das das ausschließen von Kanten schlechter als der Normale Ansatz ist. Das fixieren der Hülle sowie die alternative Kantenbedingung liefern ähnliche Ergebnisse. Es ist aber klar erkennbar das Gurobi keine guten Ergebnisse liefert. Bis auf die Iterativen Instanzen werden meistens nicht die 50 Prozent erreicht. In der Gesamt übersicht ist zu erkennen das das fixieren der Hülle leicht besser ist als der normale Solver.

E. Anzahl Lösungen

Knoten Anzahl	Delaunay Flips	Delaunay	Greedy	Iterative	Random
30_1	1	1	1	1	1
30_2	1	1	1	1	1
40_1	1	1	1	1	1
40_2	1	1	1	1	1
50_1	1	2	1	1	1
50_2	1	1	1	1	1
60_1	1	1	1	1	1
60_2	1	1	1	1	1
70_1	1	1	1	1	1
70_2	1	1	1	1	1
80_1	1	1	1	1	1
80_2	1	1	1	1	1
90_1	1	-1	1	1	1
90_2	1	-1	1	1	1
100_1	1	-1	-1	1	1
100_2	1	-1	-1	1	1
110_1	-1	-1	-1	1	1
110_2	-1	-1	-1	1	1
120_1	-1	-1	-1	1	1
120_2	-1	-1	-1	1	1

Tabelle E.1.: In dieser Tabelle werden die Anzahl der Lösungen für die verschiedenen Ansätze dargestellt. In der ersten Spalte werden die Anzahl der Knoten der Instanz angegeben. Hierbei wurde jede Knotengröße zwei mal getestet. In den anderen Spalten sind jeweils die Werte für eine Instanz gegeben. Der Wert -1 bedeutet, dass nach einer Stunde ein Timeout erreicht wurde und somit keine Lösung gefunden werden konnte. Auffällig ist, dass es nur eine einzige Instanz gibt, welche zwei Lösungen besitzt.

F. größerer Timeout

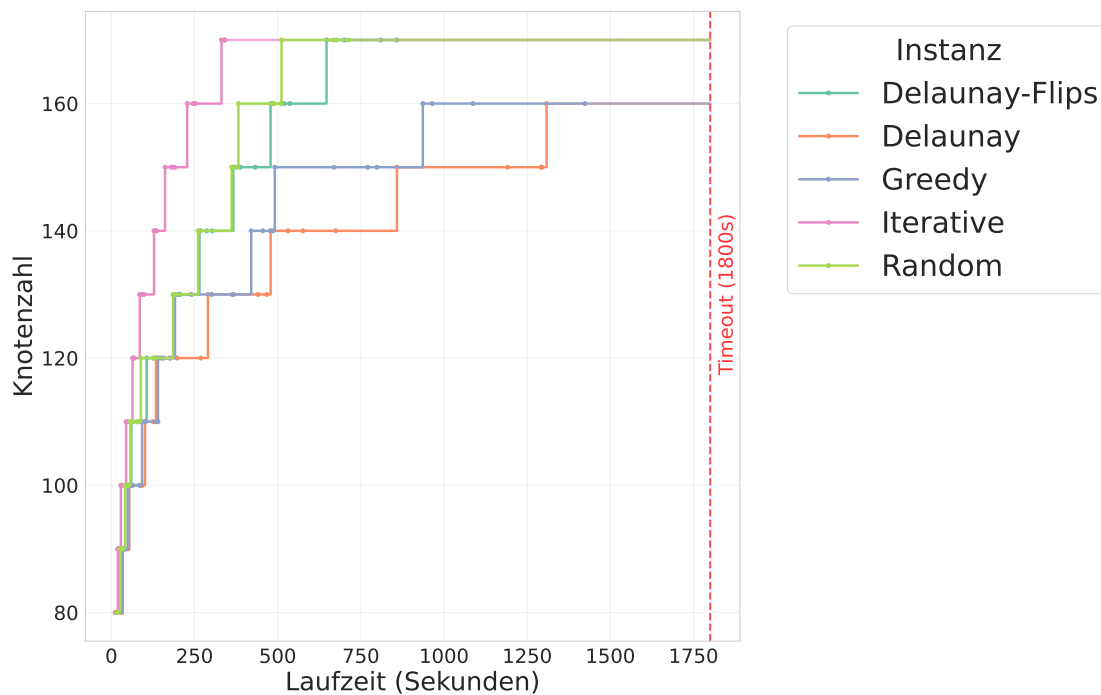


Abbildung F.1.: In diesem Experiment wurde das Zeitlimit von 5 Minuten auf 30 Minuten erhöht. Auf der Y-Achse wird in diesem Diagramm die Anzahl der Knoten angezeigt. Das bedeutet der Solver muss alle vier Instanzen der aktuellen Knotengröße lösen damit die Linie um eine Einheit nach oben versetzt wird. Es ist zu erkennen das der Solver Kantengrößen bis zu **170** Knoten Lösen Konnte. Allerdings nur für Iteraitv, Random und Greedy. Für Delaunay und Greedy konnten nur Knoten der gröÙe **160** gelöst werden.

G. Ja/Nein getrennt

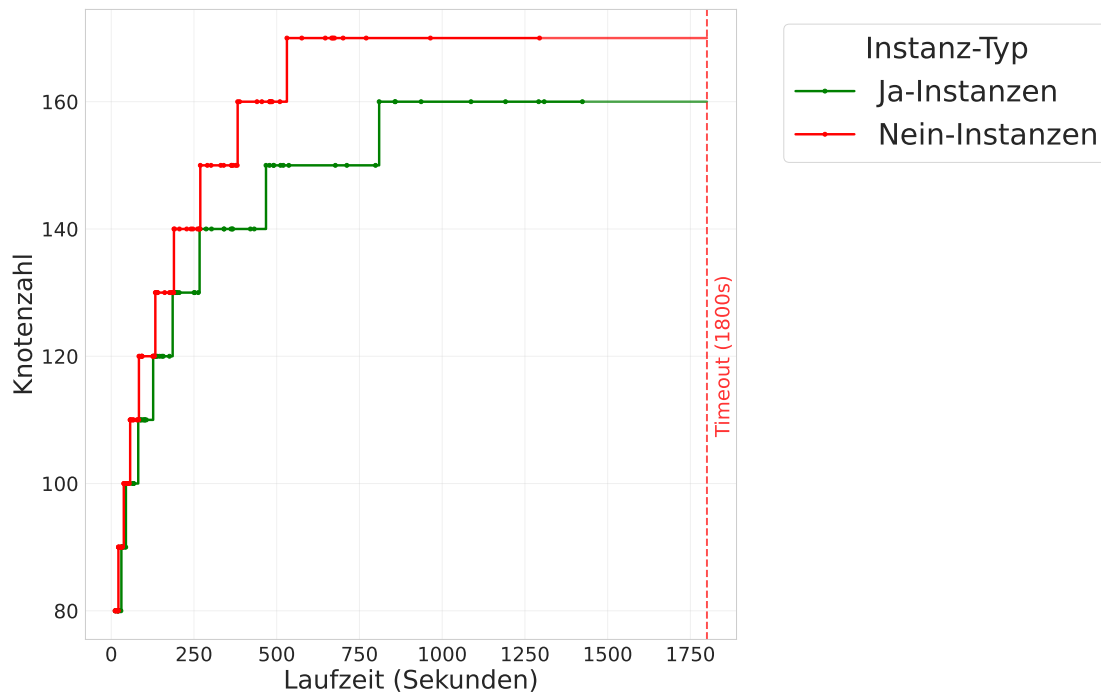


Abbildung G.1.: In diesem Experiment wurden die Daten aus Kapitel F genutzt um zu untersuchen ob der Solver bei Ja-Instanzen oder Nein-Instanzen schneller ist. Dafür wurde das gleiche Diagramm wie in Abbildung F.1 verwendet, nur dass die Ja-Instanzen und Nein-Instanzen gruppiert betrachtet werden und nicht die Instanztypen. Auch müssen hier alle 10 Instanzen der Knotengröße gelöst werden damit die Linie um eine Einheit nach oben versetzt wird. Die 10 setzt sich aus fünf Instanzen und jeweils zwei Ja-Instanzen bzw. Nein-Instanzen zusammen. Es ist zu erkennen dass die Nein-Instanzen bis zu **170** Knoten schaffen und die Ja-Instanzen nur bis zu **160** Knoten. Es lässt sich auch klar erkennen dass die Nein-Instanzen insgesamt schneller sind.

H. Zielfunktion

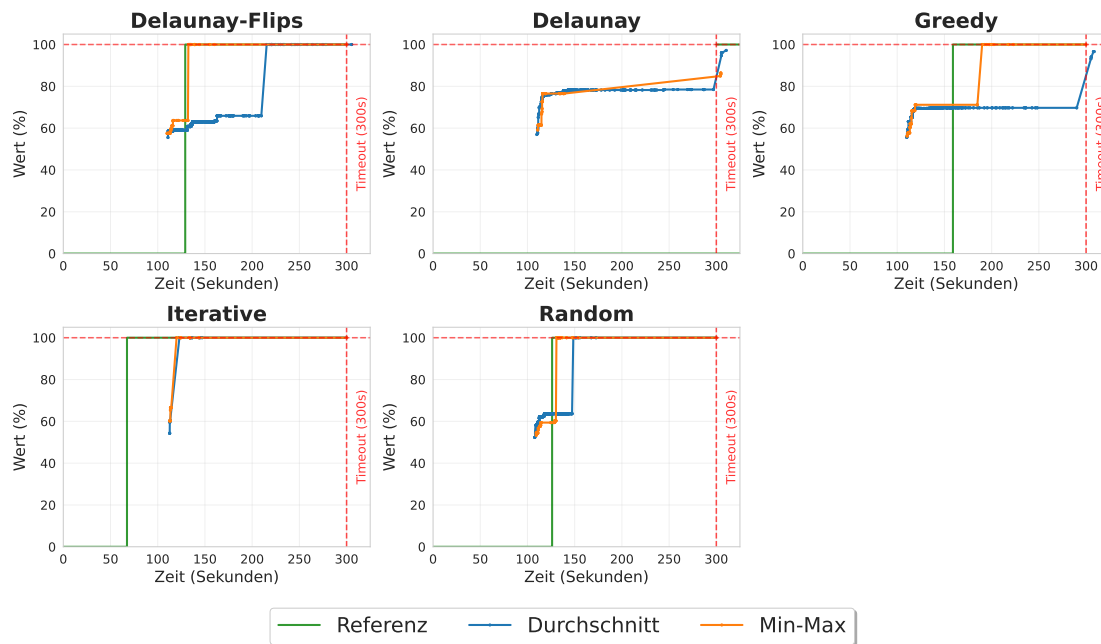


Abbildung H.1.: In diesem Experiment wird eine Greedy Ja-Instanz mit 120 Knoten als Optimierungsproblem betrachtet. Als Referenz wird der Solver aus Abschnitt 6.6 verwendet. Dieser betrachtet die Instanz als Entscheidungsproblem. Es ist zu sehen, dass der Min-Max-Ansatz in den meisten Fällen bessere Ergebnisse erzielt als der Durchschnittsansatz. Nur bei Delaunay erreichen beide Ansätze nicht die vollen 100 Prozent. Der Durchschnittsansatz erzielt bei Delaunay jedoch höhere Werte als Min-Max. Der Referenzsolver liefert bei Delaunay kein Ergebnis. Ein klares Muster, welcher Ansatz insgesamt am besten abschneidet, ist nicht zu erkennen, da in jeder Instanz unterschiedliche Ansätze vorne liegen. Der Durchschnittsansatz erzielt, wie erwähnt, bei Delaunay die besten Resultate. Der Referenzsolver ist bei Iterative und Greedy am schnellsten. Min-Max ist fast immer schneller als der Durchschnittsansatz und in drei Fällen mindestens genauso schnell wie der Referenzsolver.

I. Theoretische Fixierung von Kanten

FA: Timeout nicht anzeigen

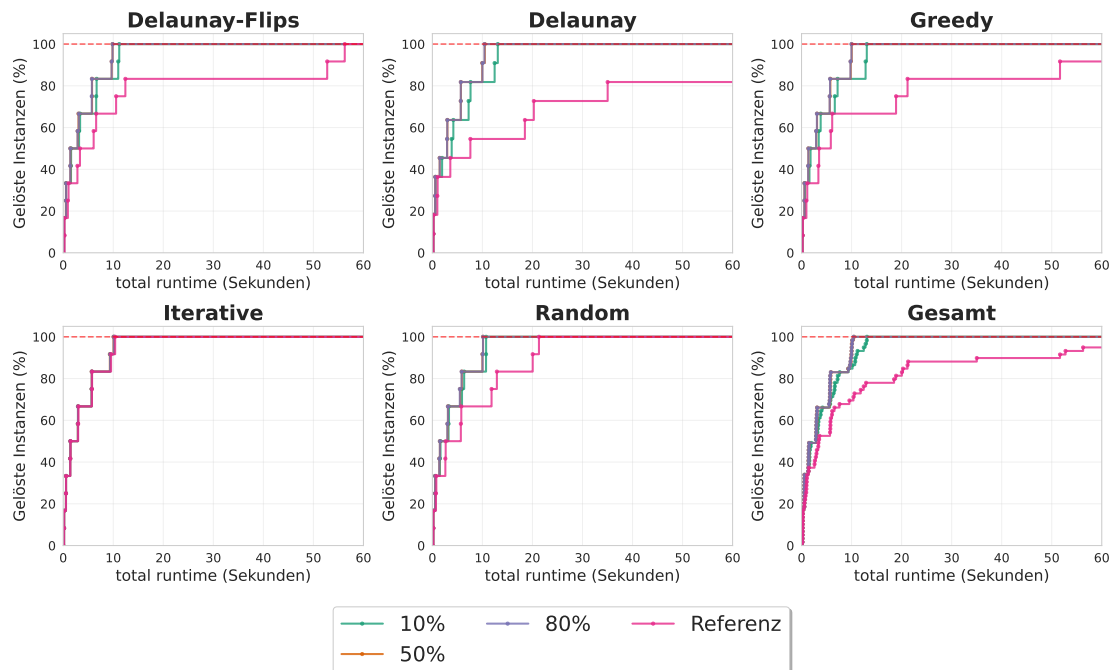


Abbildung I.1.: In diesem Experiment wird die theoretische Fixierung von Kanten untersucht. Für das Experiment wurde OrTools mit Grad und Überschneidungsbeschränkung als Referenz und zusätzlich OrTools mit 10, 50 und 80 Prozent der benötigten Kanten fixiert verwendet. In dem Diagramm wurde der betrachtete Zeitraum auf 60 Sekunden begrenzt da hier alle Relevanten Informationen zu sehen sind. Es ist klar zu erkennen das normale Solver mit Abstand am langsamsten ist. Wie zu erwarten, ist der Solver, bei dem nur 10 Prozent der Kanten fixiert wurden, langsamer als die beiden anderen. Überraschend ist, dass es bei keiner Instanz einen Unterschied macht, ob 50 oder 80 Prozent der Kanten fixiert wurden. Die beiden Solver sind genau gleich schnell.